

LEXI: Priority-First Service Orchestration

Xuan-Bach Le¹ ^{*}, Phat T. Tran-Truong¹ , Duy Nguyen² , and Ngoc Khanh Tran Nguyen³ 

¹ Ho Chi Minh City University of Technology (HCMUT), VNU-HCM, Vietnam
`{lexuanbach, phatttt}@hcmut.edu.vn`

² University of Economics Ho Chi Minh City, Vietnam
`duynguyen.726202321440@st.ueh.edu.vn`

³ University of Science, VNU-HCM, Vietnam
`22120378@student.hcmus.edu.vn`

Abstract. A service deployed across cloud, edge, and serverless nodes must balance latency, data exposure, carbon, and cost. Operators may specify these objectives in a strict order, such as satisfying a latency SLO before optimising the remaining criteria. Weighted sums approximate this order only for deployment-specific weights and can change decisions when an objective’s numerical scale is misestimated. We present LEXI, a continuum orchestrator that applies a thresholded lexicographic selection rule to predictions from an off-policy counterfactual outcome model. The model assumes that objectives decompose across stages. The rule applies to any ordered bundle. We instantiate it as $\text{SLO} \succ \text{privacy} \succ \text{carbon} \succ \text{cost}$, using a simple topological privacy proxy only to distinguish placements already tied on the SLO. We prove scale invariance for strict selection, optimality within the threshold bands, and a top-priority tracking bound. In a 15-seed discrete-event simulation with 7 nodes, a 6-stage DAG, and 40 candidate placements per decision, LEXI requires no objective weights and obtains **1.46%** SLO violations with PII exposure **0.05**. These outcomes reach the same target region as hand-calibrated and SLO-constrained weighted sums. An untuned sum reaches 23% SLO violations and 1.94 PII exposure, and only one of 84 tested weightings meets the target. When privacy is under-scaled by $10\times$, exposure rises to **1.85** and **1.84** of two stages for the tuned and constrained baselines, respectively, while LEXI is unchanged. A rerun with UK-grid carbon and Azure Functions load traces preserves these comparisons. Our evidence covers one simulated application at the stated scale under the additive no-interference assumption.

Keywords: Service orchestration · Cloud-edge-serverless continuum · Lexicographic optimisation · Multi-objective decision-making · Scale invariance · Carbon-aware placement.

* X.-B. Le and P. T. Tran-Truong contributed equally to this work.

✉ Corresponding author: Xuan-Bach Le, `lexuanbach@hcmut.edu.vn`.

Extended version of the ICSOC 2026 paper. Artifact: <https://github.com/lexuanbach/lexi>.

1 Introduction

Modern service workloads span regional cloud, on-premises edge, and serverless resources [5,35,38,22]. Placement determines the latency, data exposure, carbon, and cost of each stage. Service computing [6] and carbon-aware systems [32,43] treat these as first-class objectives, but many placement policies still reduce them to one heuristic or a weighted sum.

Operators may instead specify a strict priority order. For example, a latency service-level objective (SLO) should be satisfied before privacy, carbon, or cost is optimised. A weighted sum $\sum_k w_k \text{obj}_k$ assigns exchange rates between objectives and cannot represent this order for arbitrary objective values: a sufficiently large lower-priority improvement can offset a small higher-priority loss. Widely separated weights can approximate the order over a bounded deployment, but the suitable region is small and depends on objective scales. Only one of the 84 weightings tested here meets the priority target, and an untuned sum violates the SLO on 23% of requests (Sect. 9). Changes in units or operating ranges can also alter the induced priority without changing the operator’s intent.

We begin each LEXI decision with predicted outcomes for the available placements (Fig. 1). It retains the candidates that are best on the first objective within a declared band, then repeats the comparison on each lower-priority objective until one candidate remains. A counterfactual outcome model supplies the predictions. Under additive no-interference (Assumption 1), per-stage observations from the executed placement can be recombined to estimate unexecuted placements that share the same nodes [7,31]. The evaluated controller makes no deliberate exploratory migration. Executed placements update the corresponding per-node models. Predictions elsewhere additionally require model adequacy and coverage of the relevant node-feature combinations [27].

We evaluate the order $\text{SLO} \succ \text{privacy} \succ \text{carbon} \succ \text{cost}$. Privacy is represented by the number of PII-tagged stages placed on publicly reachable nodes. This is a topological preference rather than a compliance certificate: it omits residency, encryption, transit security, and access-policy misconfiguration [11]. Because privacy is second in the order, it only separates placements already tied on the SLO. Section 3.4 states the proxy’s scope in detail. The selection rule and theory depend on coordinate order rather than on this particular interpretation. Another ordered objective can occupy the same position.

We make three contributions. First, we formulate continuum placement as priority-first control and separate the counterfactual predictor from the lexicographic selector (Sects. 3–5). A numerical walkthrough in Sect. 4 traces this interface for one request. Second, we prove thresholded-lexicographic optimality over the candidate menu, invariance of strict selection to strictly increasing per-objective transformations, a top-priority estimation-error bound under additive no-interference, and linear decision complexity (Sect. 6, full proofs in Sect. 7). Third, we compare LEXI with calibrated, constrained, and scale-robust scalarisers across the main simulation, objective-scale perturbations, model misspecification, real-trace inputs, and additional stress tests (Sects. 8–12).

We run all experiments in a discrete-event simulator rather than a physical testbed. The primary configuration contains one application, 7 nodes across 3 regions, a 6-stage DAG, and 40 candidate placements per decision. A second configuration and scaling runs to $K \leq 30$ stages and $C \leq 2000$ candidates probe sensitivity, but no multi-application or multi-tenant deployment is evaluated. Public traces provide carbon intensity and arrival load. The remaining simulator parameters are authorial choices documented in Sect. 8.1. The experiments do not measure orchestration overhead, migration cost, control-plane delay, or physical-network jitter.

2 Background and Preliminaries

The cloud-edge-serverless continuum. A *continuum* is a mixed fleet of compute locations [5,35,38]: regional cloud (strong compute, high round-trip time (RTT), public), on-prem edge (weak compute, low RTT when local, private), and serverless (cheap, elastic, public). A *service DAG* is a set of stages with data dependencies, drawn as a directed acyclic graph (e.g. `frontend` $\rightarrow$ `auth` $\rightarrow$ `{search,recommend}` $\rightarrow$ `order` $\rightarrow$ `pay`). Some stages carry PII. A *placement* assigns each stage to a node. It fixes four things: the critical-path latency (the slowest path through the graph), whether the PII stages sit on a private node, the carbon (energy times grid intensity, which varies by node and time), and the cost.

Multi-objective and lexicographic decision-making. A multi-objective problem returns a cost vector $\mathbf{g} \in \mathbb{R}^d$, where lower is better on each coordinate. The dominant method, *scalarisation*, collapses the vector into one number $\sum_k w_k \mathbf{g}_k$ [33,41] and so sets an exchange rate between goals. But our goals are *ranked* (SLO before privacy before carbon before cost), and no finite weight vector expresses this: a large low-priority saving can always outweigh a small high-priority loss. *Lexicographic* optimisation [19,36] instead compares two vectors on the first coordinate where they differ. A *thresholded* rule relaxes this with a per-goal slack η_k : it keeps every candidate within η_k of the best value on goal k before it moves to goal $k+1$.

Off-policy estimation. *Counterfactual* reasoning [31,7] estimates the outcome of an action that was *not* taken, using data from actions that were. Suppose each goal is a sum of per-(stage, node) terms (an explicit structural assumption, Sect. 6) and the candidates share the fleet. Then one executed request identifies the per-assignment terms, and so the outcomes of *many* un-executed candidates. This is *off-policy* learning [13] (learning from decisions already made), and it lets LEXI learn without exploratory migrations.

3 The LEXI Model and Controller

We construct LEXI from four dependent mechanisms: the placement problem, the transformation from physical outcomes to a normalised priority vector, the selec-

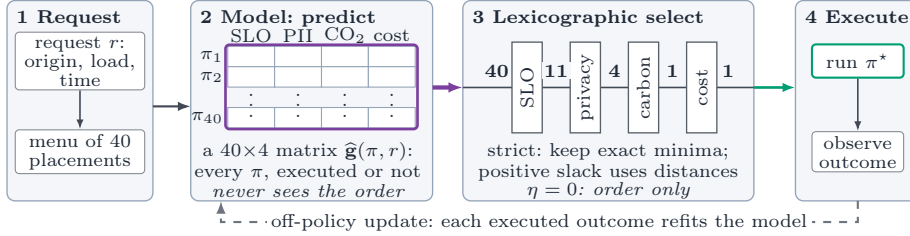


Fig. 1. LEXI’s decision loop: the model predicts every objective for every candidate (purple: the only interface). Strict selection reads order only. Positive slack also reads within-coordinate distances (counts: Table 1).

tion rule over that vector, and the counterfactual outcome model that supplies it. At each step, we distinguish an *estimate* from a *decision*, then make their interface explicit in Sect. 3.6. Figure 1 summarises the resulting loop, Sect. 4 traces one request numerically, and Sect. 5 specifies the controller.

3.1 The continuum, the service DAG, and a placement

Definition 1 (Continuum). A continuum is a set of nodes $\mathcal{N} = \{1, \dots, M\}$. Each node has a tier in $\{\text{cloud}, \text{edge}, \text{serverless}\}$, a region, a per-demand compute time, and an origin-dependent RTT that is low when the origin matches the node’s region. Each node also has a queueing sensitivity, a per-invocation synthetic cost, an energy per reference demand, a time-varying carbon intensity $I_n(t)$, and a privacy tier (edge is private, cloud and serverless are public).

Definition 2 (Service DAG, placement, physical objectives). A service DAG has stages $1, \dots, K$ with data dependencies, and each stage has a demand and a PII tag. A placement $\pi : \{1, \dots, K\} \rightarrow \mathcal{N}$ assigns stages to nodes. Take a request r with origin region, time t , and load ℓ . Executing π then yields the physical objective vector $\mathbf{g}^{\text{raw}}(\pi, r) \in \mathbb{R}^4$, where all four coordinates are lower is better. Latency is the critical-path sum of the per-stage RTT, compute, and queueing terms. Privacy is the number of PII-tagged stages on a public node (Sect. 3.4). With energy in Wh and intensity in $\text{gCO}_2\text{eq/kWh}$, carbon is $\sum_s \text{energy}_{\pi(s)}(\text{demand}_s/20)I_{\pi(s)}(t)/1000$ grams, and cost is $\sum_s \text{cost}_{\pi(s)}$ in synthetic units. Churn is the number of stage assignments changed from the previous request and is reported but not ranked.

3.2 From physical outcomes to a normalised priority vector

The four physical objectives use different units (milliseconds, a stage count, grams, and dollars). They are mapped to a *normalised priority space* $[0, 1]^4$. The top objective and the remaining objectives use different transformations.

The top coordinate is a hinge rather than a race. Objective 0 is the SLO hinge

$$\mathbf{g}_0 = \min\left(1, \frac{\max(0, \text{latency} - \text{SLO})}{\rho}\right), \quad (1)$$

which is exactly 0 once the SLO is met and rises with the overrun over a scale ρ . This encodes the operator’s actual intent: latency is a *threshold* requirement rather than something to minimise without limit. The consequence is structural and drives everything downstream — every placement that meets the SLO is *tied* at $\mathbf{g}_0 = 0$. The top priority typically returns a *plateau* of candidates rather than a single winner, and the second objective is what actually separates them. Eq. (1) also saturates: two placements that overrun by more than ρ tie at $\mathbf{g}_0 = 1$ and are likewise separated further down. With the 260 ms SLO and $\rho = 60$ ms, every latency at or above 320 ms is on this plateau.

The remaining coordinates are divided by a declared magnitude and clamped. Each of privacy, carbon, and cost is divided by an operator-declared typical magnitude and clamped to $[0, 1]$ (in our configuration: 2 stages, 2.5 g, 5 cost units). Clamping keeps every objective bounded, which the analysis of Sect. 6 needs. The divisors and ρ are strictly increasing maps. By Prop. 2 they set only the *width* of the lower-priority slack bands: while no candidate saturates a clamp they leave the chosen placement and its ranking unchanged. Consequently, a weighted sum depends on correctly calibrated weights, whereas strict LEXI selection is unchanged by these divisors while no candidate saturates the clamp. The unit-bearing controller parameters are the optional carbon slack (which the quantile rule of Sect. 5 makes unit-free) and the SLO margin δ , which is a millisecond quantity on a millisecond objective.

3.3 The priority order and its selection rule

Definition 3 (Priority order and its selection rule). *The operator specifies a strict priority order over the coordinates — here latency-SLO $\succ$ privacy $\succ$ carbon $\succ$ cost — realised directly on the normalised objective coordinates. The thresholded-lexicographic optimum with slack $\boldsymbol{\eta} = (\eta_0, \eta_1, \eta_2)$ keeps, for $k = 0, 1, 2$ in turn, the candidates within η_k of the current best on objective k , then breaks the final tie on cost. With $\boldsymbol{\eta} = \mathbf{0}$ this is the strict lexicographic optimum (Fig. 1), using only the order of each objective’s values, never numeric weights. For a convergence plot only, we also use the bounded surrogate $L(\mathbf{g}) = \sum_k \beta^{-k} \mathbf{g}_k$ ($\beta=8$), an ε -lexicographic aggregate (granularity in Sect. 6) rather than the selection rule.*

Three properties of Def. 3 are worth making explicit, because they are what the rule buys and what it costs.

1. *Strict selection consumes only order.* With $\boldsymbol{\eta} = \mathbf{0}$, the selector never adds, multiplies, or compares values *across* coordinates. It only ever asks “which candidates attain the minimum of coordinate k among those still surviving”.

This is the source of the scale-invariance of Prop. 2. Positive additive slack instead uses within-coordinate distances and must be rescaled with a positive linear change of units.

2. *Lower priorities act only on ties.* Objective k is consulted only on the set that objectives $0, \dots, k-1$ have failed to separate. A lower-priority objective can therefore cause no higher-priority regression under strict selection. With positive slack it may choose within the explicitly tolerated higher-priority band.
3. *Ties must actually occur for lower priorities to matter.* Whether a coordinate does any work depends on how tie-prone the coordinates above it are. Here the top two are strongly tie-prone — $\mathbf{g}_0$ is a hinge that collapses every feasible placement to 0, and $\mathbf{g}_1$ is integer-valued — so privacy routinely separates an SLO-tied plateau, whereas carbon is near-continuous and usually resolves to a singleton, leaving cost to break only exact carbon ties. Sect. 4 shows this happening on a real step, and Sect. 9 quantifies it over the full run.

Intuition on four real candidates. The rule filters rather than scores: each level keeps the (near-)best candidates on one column and passes only those down. A value on a lower column is never weighed against a value on a higher one. Take four of the 40 candidates of the decision traced in Sect. 4 (seed 0, step $t = 904$), with predicted normalised vectors $(\mathbf{g}_0, \mathbf{g}_1, \mathbf{g}_2, \mathbf{g}_3)$: #18 (0.076, 0, 0.673, 0.888), #6 (0, 1, 0.166, 0.216), #20 (0, 0, 0.387, 0.608) and #21 (0, 0, 0.314, 0.592).

- (a) *The hinge ties every SLO-feasible placement.* Level 0 drops #18, predicted 4.5 ms over the SLO. #6 is predicted 7 ms faster than #21, but both meet the SLO. The hinge ties them at 0: being faster than required earns nothing.
- (b) *Privacy only breaks SLO ties.* Level 1 drops #6, which exposes both PII stages. Privacy acts only on the set the SLO has already tied. It could not have rescued #18.
- (c) *A band keeps near-ties.* Carbon picks #21 ($0.314 < 0.387$). A band $\eta_2 = 0.08$ would instead keep #20 as a near-tie ($0.387 \leq 0.314 + 0.08$) and defer to the final tie-break — cost, or the hysteresis hold of Sect. 5 — which here returns #21 again (cost $0.592 < 0.608$, and #21 was also the previous placement).
- (d) *A sum depends on the units.* Handed the same matrix, a sum with WS-tuned’s weights (Sect. 8) also picks #21 (score 0.030 against 0.111 for #6), but with privacy’s unit under-scaled $10\times$ it picks #6 (0.021 against 0.030). No such rescaling changes the lexicographic choice (Prop. 2).

Claims (c) and (d) hold over the full 40-candidate matrix rather than only these four rows. The exact counts, and how prediction error enters, follow in Sect. 4.

Per request LEXI predicts $\hat{\mathbf{g}}(\pi, r_t)$ for every candidate, applies Def. 3, executes and updates the model in $O(K)$. **The configuration evaluated in RQ1–RQ4 is the strict rule, $\eta = 0$,** since slack on a *threshold* objective converts directly into SLO violations. A carbon slack, an SLO safety margin and a coverage probe exist (Sect. 5) but are enabled only in RQ5–RQ7. RQ1–RQ4 do not depend on them.

3.4 Privacy coordinate and scope

The second coordinate is the one that carries the most modelling debt. We state its scope here rather than leaving it implicit.

The measurement. $\mathbf{g}_1$ counts the PII-tagged stages of a placement that sit on a node whose tier is publicly reachable (cloud or serverless), divided by the number of PII stages. It is a *topological* quantity, computed from the placement and a static per-stage data-sensitivity tag, with no observation of the data itself. In the primary configuration two of six stages (**auth**, **pay**) carry the tag. $\mathbf{g}_1 \in \{0, \frac{1}{2}, 1\}$ and the reported “PII exposure” figure is in $[0, 2]$ stages. **Being second, it is consulted only on the set the SLO has already tied: it can never override the SLO and only ever chooses among SLO-equivalent placements** (Sect. 4).

Excluded privacy properties. It is not a compliance verdict and does not attempt to be one. It ignores (i) *data residency* — which jurisdiction a node sits in, and whether moving a stage crosses a border. (ii) *encryption*, at rest or in transit, and the latency and energy that encryption costs. (iii) *transit security* — what an inter-tier hop exposes on the wire, and (iv) the *misconfiguration* failure mode that dominates real-world exposure, where the resource is nominally private but its access policy is wrong [11]. A placement that $\mathbf{g}_1$ scores as perfectly private can therefore still leak, and the proxy will not see it. We treat residency separately and only as a hard *pre-filter* (Sect. 12), which is the appropriate way to encode a constraint that is not a gradient.

Role in the evaluation. If placing PII on a public node is a known violation, then no method should be offered that placement in the first place, and if instead every publicly reachable node is treated as a leakage candidate, the judgement is unmotivated. A known violation should indeed never be a candidate for *any* method. Here, $\mathbf{g}_1$ expresses a *preference* over otherwise-acceptable placements, of the kind an operator states as “keep sensitive stages on-prem when you can”. Hard prohibitions belong in the candidate generator or a residency pre-filter rather than in a ranked objective, and that is where we put them. What $\mathbf{g}_1$ contributes to this paper is an objective with two properties the evaluation needs: it is *integer-valued*, hence genuinely tie-prone and a good stress test of tie-breaking, and it is *unit-ambiguous* — there is no natural exchange rate between an exposed stage and a gram of CO₂ — which makes it the sharpest available probe of the scale-sensitivity claim in Prop. 2. Both properties are about the *rule* rather than about privacy. Accordingly, every result in this paper that depends on $\mathbf{g}_1$ should be read as a statement about a second-ranked, integer, unit-ambiguous objective. Substituting a different such objective — a tenancy-isolation count, a licence-tier count, a jurisdiction-crossing count — would exercise the mechanism identically. Nothing in Sects. 3 to 6 refers to the coordinate’s meaning.

3.5 The counterfactual outcome model

Definition 4 (Counterfactual outcome model). *Under Assumption 1, each objective is a sum of per-(stage, node) terms, and each term depends only on that stage’s own assignment and the external request state. So executing one placement identifies those terms, and the outcome of any candidate is the corresponding sum. LEXI maintains one per-node estimator per objective online: ridge regressions for latency (features: stage demand, instantaneous load, a same-region indicator) and carbon (time-of-day harmonics), a running mean for cost, and static policy metadata for privacy, returning an estimated vector $\hat{\mathbf{g}}(\pi, r)$ for every candidate π . This claims nothing beyond Assumption 1. The misspecification ε_t (Sect. 6) is the estimate error over the near-optimal candidates.*

The four coordinates are estimated by deliberately different machinery, because they are epistemically different. Latency and carbon are genuinely uncertain and are *learned* online. Cost is deterministic given the node. A running mean converges to it exactly for any node the policy has visited — residual cost error is entirely a *coverage* artefact of a no-exploration policy, the most visible way the persistent-excitation assumption (A3) of Thm. 1 fails, which the priority-safe probe of Sect. 12 addresses. Privacy is not estimated at all: it is a deterministic function of the placement and the stage tags, read off policy metadata. Sect. 9.5 measures the realised accuracy of each.

3.6 Interface between model and rule

The prediction model and selection rule communicate through a narrow interface.

- The two components communicate through exactly one object: an $|\mathcal{C}| \times 4$ matrix $\hat{\mathbf{g}}(\cdot, r_t)$ of normalised predicted objective vectors, one row per candidate placement. Nothing else crosses the boundary.
- The model is *never* told the priority order. It minimises its own prediction error per coordinate, independently. Swapping the order is an $O(1)$ argument change with no retraining (measured in Sect. 12).
- The selector is *never* told anything about uncertainty, calibration, or units. It reads only the induced ordering of each column. Consequently the selector inherits the model’s errors but none of its parameterisation: a strictly increasing re-parameterisation of any column — a different unit, or a different divisor or hinge scale ρ while no candidate saturates the clamp — leaves the selected row unchanged (Prop. 2).
- What the model’s error *can* do is move a candidate across a band boundary: a placement whose true latency exceeds the SLO but whose predicted latency does not will survive level 0 and compete for the choice. This is the only channel by which prediction error harms the decision, and it is exactly the quantity ε_t that Thm. 1 bounds. Both the mechanism and its magnitude are visible in the walkthrough of Sect. 4 and measured in Sect. 9.5.

4 A Worked Decision: From Signals to a Placement

The model predicts an outcome vector for every candidate, and the selector consumes those vectors one priority at a time. A concrete decision makes this interface easier to see than the component definitions alone. The following trace shows how the controller moves from request signals to one placement.

The numbers come directly from step $t = 904$ of seed 0 in the released artifact, using the primary configuration of Sect. 8 after 904 online updates. This step is useful because it exposes all four priority levels in a single trace.

4.1 Stage 1 — the request signals

The controller receives a request r_{904} with three exogenous signals: origin region 2, instantaneous load $\ell = 0.765$, and wall time $t = 904$ (time-of-day fraction 0.767). Nothing else about the request is used. The SLO is 260 ms, the DAG is the six-stage pipeline `frontend` $\rightarrow$ `auth` $\rightarrow$ `{search, recommend}` $\rightarrow$ `order` $\rightarrow$ `pay`, and `auth` and `pay` carry the PII tag. The candidate menu $\mathcal{C}$ holds $|\mathcal{C}| = 40$ placements, fixed in advance (Sect. 8.1).

The origin matters because RTT is region-dependent: with origin 2, the three region-2 nodes (E2, C2, S2, i.e. `edge_C`, `cloud_C`, `srv1_C`, node ids as in Table 2) are reachable at their low intra-region RTT while everything else pays the inter-region RTT. This signal changes the objective values from step to step, which is why the controller estimates them for each request.

4.2 Stage 2 — the counterfactual outcome model predicts every candidate

The model (Def. 4) is asked for $\widehat{\mathbf{g}}(\pi, r_{904})$ for *all* 40 candidates, including the 39 that will not be executed. It does this without having tried any of them: each per-node estimator was fitted on the per-stage signals of the placements actually executed in steps 0–903, and under Assumption 1 a candidate’s objective is just the corresponding sum (critical path, for latency) over its own per-(stage, node) terms. The output is a 40×4 matrix of normalised predicted objective vectors. This matrix is the interface to the selector (Sect. 3.6).

Prediction quality at this step is representative: over the 40 candidates the mean absolute latency error is a few milliseconds, but it is not zero, and Stage 3 shows where that matters.

4.3 Stage 3 — thresholded lexicographic selection, level by level

The selector now consumes the matrix one column at a time. With $\boldsymbol{\eta} = \mathbf{0}$ (strict), each level keeps the candidates attaining the current minimum and discards the rest. Table 1 lists the 11 candidates that survive level 0. Its “pred.” latency is what the counterfactual model predicted. The “true” latency is the noise-free simulator value, shown for exposition only, as the selector never sees it. The normalised columns are what the selector actually reads, and the last column gives the level at which each candidate is discarded.

Level 0, the SLO hinge: $40 \rightarrow 11$ candidates. The minimum predicted hinge is $\mathbf{g}_0 = 0$ — some candidates are predicted to meet the 260ms SLO — so the surviving set is every candidate with $\hat{\mathbf{g}}_0 = 0$, which is 11 of 40. The top priority therefore produces an eleven-way tie instead of a single winner. Equation (1) is a threshold. Every predicted-feasible placement scores exactly 0. This plateau behaviour (Sect. 3.2) gives the second priority a concrete role.

Prediction error is also visible here. Of those 11 survivors, only 9 truly meet the SLO. Candidates 24 and 33 (true latency 268.2ms and 264.8ms) were admitted because the model under-predicted them. This admission error is the channel represented by ε_t in Thm. 1. Both candidates are discarded at level 1, and Sect. 9.5 measures how often such errors change the final decision.

Level 1, privacy: $11 \rightarrow 4$ candidates. Among the eleven SLO-tied survivors the minimum predicted exposure is $\mathbf{g}_1 = 0$. The four candidates that keep both PII stages on a private edge node survive: $\{2, 19, 20, 21\}$ (Table 1). The other seven are discarded.

This step instantiates the scope stated in Sect. 3.4: privacy is consulted only among candidates tied by the SLO. Here it reduces an eleven-way SLO tie to four candidates. At higher load, fewer candidates may reach level 1. If none meets the SLO, level 0 first minimizes the overrun and privacy then distinguishes only candidates with the same overrun.

Level 2, carbon: $4 \rightarrow 1$ candidate. Carbon is near-continuous. Exact ties are rare: among $\{2, 19, 20, 21\}$ the predicted carbon is 1.683, 1.190, 0.968 and 0.784 g, and candidate 21 wins outright.

Level 3, cost: not reached. Level 2 already returned a singleton. The cost tie-break does not fire. This is the common case in our configuration, and it is a property of the objectives rather than of the rule: the two tie-prone coordinates (a saturating hinge, an integer count) sit at the top, and the two near-continuous ones sit below. Cost would decide between candidates tied on carbon, but such ties are rare in this configuration. Candidate 21 is also the cheapest of the four, at \$2.96.

The result. LEXI executes candidate 21: **auth** and **pay** on the private edge node E2, the four non-PII stages on the cheap, low-carbon, region-local serverless node S2. Noise-free objectives: latency 235.5ms (inside the SLO), PII exposure 0 of 2, carbon 0.785 g, cost \$2.96. Computing the entire trace took well under 0.1 ms (Prop. 3).

4.4 Stage 4 — execute and update off-policy

The chosen placement is executed and its per-stage realised signals (latency, carbon, cost per stage) are fed back to the per-node estimators. Because candidate 21 touches E2 and S2, and those two nodes appear in many other candidates, this single execution sharpens the predictions for a large slice of $\mathcal{C}$ at the next

Table 1. Step $t=904$ (seed 0), verbatim from the artifact: all eleven level-0 survivors. **Bold:** the choice. †: admitted at level 0 by prediction error.

#	Placement	Latency (ms)		Normalised $\hat{\mathbf{g}}$ (what the selector reads)				Elim.
		pred.	true	$\mathbf{g}_0$	$\mathbf{g}_1$	$\mathbf{g}_2$	$\mathbf{g}_3$	
2	E2/E2/E2/E2/E2/E2	252.5	251.9	0	0.0	0.673	1.000	L2
4	C2/C2/C2/C2/C2/C2	222.7	249.0	0	1.0	0.395	0.600	L1
6	S2/S2/S2/S2/S2/S2	228.9	228.8	0	1.0	0.166	0.216	L1
9	E2/S2/C2/C2/S2/S2	215.6	220.5	0	1.0	0.309	0.532	L1
19	C2/E2/C2/C2/C2/E2	230.3	245.8	0	0.0	0.476	0.848	L2
20	S1/E2/S1/S1/S1/E2	259.1	259.0	0	0.0	0.387	0.608	L2
21	S2/E2/S2/S2/S2/E2	235.7	235.5	0	0.0	0.314	0.592	—
24 [†]	C0/C2/S2/S1/C2/C0	256.2	268.2	0	1.0	0.371	0.496	L1
31	S1/S2/E2/C2/S2/C2	252.4	257.1	0	1.0	0.388	0.536	L1
33 [†]	C2/S1/S1/C0/E2/E2	259.3	264.8	0	0.5	0.496	0.738	L1
36	S1/C2/C0/E2/S1/E2	255.0	259.9	0	0.5	0.539	0.738	L1
<i>Survivors: 11 after level 0, {2, 19, 20, 21} after level 1, {21} after level 2; level 3 not reached.</i>								

step — with no exploratory migration. That is the whole content of “off-policy”: the update is driven by the decision that was actually taken, and it informs the ones that were not.

4.5 Comparison with weighted-sum decisions

The same 40×4 matrix, handed to a weighted sum, gives a different answer depending entirely on the weights. We ran both weightings of Sect. 8 on this exact matrix:

- The **untuned** sum ($w = (0.30, 0.12, 0.30, 0.28)$) selects candidate 6, **S2**×6 — and thereby exposes **both** PII stages. Candidate 6 is better than the winner on three of the four coordinates (latency 228.8 vs 235.5 ms, carbon 0.416 vs 0.785 g, cost \$1.08 vs \$2.96). With these weights, the \$1.88 cost saving and 0.37 g carbon saving outweigh the penalty for exposing two stages. This behavior follows from the encoded exchange rates.
- The **tuned** sum ($w = (0.85, 0.10, 0.00, 0.05)$, the calibrated weights of Sect. 8) selects candidate 21, the same placement chosen by LEXI. The SLO-constrained sum Constr-WS does so as well. This agreement illustrates the nominal comparison: a well-tuned scalariser can reproduce the priority order. Sections 9 and 9.3 measure the separation when tuning is unavailable, deployment conditions change, or an objective’s scale is misestimated.

LEXI reaches candidate 21 from the declared order, without a weight vector or an exchange rate between carbon and exposure. The monotone transformations preserve that order, and the counterfactual model avoids an exploratory placement for this decision.

5 Controller Mechanisms in Detail

LEXI runs one decision per request (Algorithm 1). It first predicts every candidate’s objective vector with the counterfactual outcome model. It then selects by

Algorithm 1: LEXI: one control step for request r_t

Input: candidate set $\mathcal{C}$, outcome model $\mathcal{M}$, slack η , previous placement π_{t-1}

```

1  $\hat{\mathbf{g}}(\pi, r_t) \leftarrow \mathcal{M}.\text{PREDICT}(\pi, r_t)$  for all  $\pi \in \mathcal{C}$  // counterfactual, normalised
2  $I \leftarrow \mathcal{C}$ 
3 for  $k \leftarrow 0$  to 2 do
4    $b_k \leftarrow \min_{\pi \in I} \hat{\mathbf{g}}_k(\pi, r_t)$  // best on objective  $k$ 
5    $I \leftarrow \{\pi \in I : \hat{\mathbf{g}}_k(\pi, r_t) \leq b_k + \eta_k\}$  // keep the  $\eta_k$ -band
6 if  $\pi_{t-1} \in I$  then  $\hat{\pi} \leftarrow \pi_{t-1}$  // stability: no needless migration
7
8 else  $\hat{\pi} \leftarrow \arg \min_{\pi \in I} \hat{\mathbf{g}}_3(\pi, r_t)$  // final tie-break on cost
9
10 execute  $\hat{\pi}$ ; observe outcome  $\mathbf{g}(\hat{\pi}, r_t)$  and per-stage signals
11  $\mathcal{M}.\text{UPDATE}(\hat{\pi}, r_t, \text{signals})$  // off-policy: informs many un-taken
   candidates
12 return  $\hat{\pi}$ 

```

thresholded lexicographic priority, on the objective *order* and with no numeric weights, subject to a stability rule. Finally it executes the choice and updates the model off-policy. This section takes each mechanism in turn and states what it does, what it costs, and whether it is on by default.

5.1 Mechanism 1: counterfactual prediction (always on)

PREDICT aggregates the per-node estimators over a candidate’s stages: a critical-path sum for latency, a stage-wise sum for carbon (time-of-day harmonics per node), a running mean for cost, and policy metadata for privacy. It then maps the result into the normalised priority space of Sect. 3.2. UPDATE feeds the *executed* placement’s per-stage signals back into the node-level estimators. Those nodes recur across *many* candidates. One execution sharpens a large slice of $\mathcal{C}$ — the mechanism illustrated at Stage 4 of Sect. 4. A weak nominal prior (an operator spec sheet: nominal per-tier compute, RTT, price, and grid intensity) seeds the model. The initial decisions are usable and no exploratory migration is ever required.

The cost of this mechanism is the assumption it rests on. Additivity across stages (Assumption 1) is what makes one execution informative about un-taken candidates. When it fails, the predictions degrade, and Sect. 9.4 measures by how much.

5.2 Mechanism 2: thresholded selection (always on, $\eta = 0$ by default)

The k -loop implements Def. 3: first the SLO-hinge band, then privacy, then carbon, and finally minimise cost. Slack lives on the *lower* priorities only. The SLO hinge ($\eta_0=0$) and privacy ($\eta_1=0$) stay strict. Only carbon may carry a band ($\eta_2>0$), and even that is off in the default strict configuration.

We keep $\eta_0 = 0$ deliberately. Slack on a *threshold* objective is not a mild relaxation: it admits placements that overrun the SLO into the privacy search, and since privacy then selects among them freely, a nonzero η_0 converts directly into SLO violations. The top priority is instead protected by an explicit margin (Mechanism 4), which reserves headroom rather than tolerating overrun. Slack on η_1 is likewise off by default, and Sect. 12 shows why it matters: scale-invariance is a property of *strict* selection, and a nonzero privacy slack is the one setting that measurably degrades it.

5.3 Mechanism 3: stability by hysteresis (on by default, no effect when $\eta = 0$)

Re-deciding per request would migrate stages whenever two near-equivalent candidates swap places. The hysteresis rule therefore retains the previous placement whenever it survives every band. With strict slack the bands are singletons or exact ties. The rule almost never fires, with a carbon band it converts the slack into stability, cutting churn from 1.35 to 1.14 stages per request at no SLO cost (Sect. 9.6). A *quantile* auto-slack sets η_2 from the candidates’ own carbon interquartile spread, which makes the knob unit-free and therefore preserves the scale-invariance argument.

Hysteresis has a stated theoretical price, and we do not hide it: when the rule retains an incumbent, Prop. 1 weakens from full lexicographic optimality to optimality on objectives 0–2 with cost-optimality only up to the hold.

5.4 Mechanism 4: the SLO safety margin (optional, off in the primary configuration)

Strict selection compares placements on their *expected* latency. A placement that just meets the SLO is still admitted even though queueing noise can push its realised latency over — and service latency has a heavy tail [12]. LEXI optionally reserves a margin δ : it first prefers placements with $\hat{\mathbf{g}}_{\text{lat}} \leq \text{SLO} - \delta$. If none qualify it falls back to those merely meeting the SLO, and then to the least overrun. It then applies the same priority order to whatever survives.

This is a chance-constraint surrogate [9], and it spends a little of the *second* priority to protect the *first* — exactly the trade the stated order sanctions. Its units are explicit: δ is a millisecond knob on a millisecond objective. Rescaling any *other* objective leaves the order unchanged (Prop. 2). Sect. 9.6 measures the trade, and Sect. 12 reports an online self-calibrating variant that sets δ from the observed latency dispersion ($k\hat{\sigma}$) rather than by hand.

5.5 Mechanism 5: online learning and the evaluation yardstick

The estimators update once per request, in $O(K)$ and independently of $|\mathcal{C}|$. The learning cost does not grow with the menu. Because the policy never explores,

coverage is not automatic: a node no candidate ever selects keeps its prior forever. Node visitation is a necessary (not sufficient) condition for the persistent-excitation assumption (A3) of Thm. 1, and Sect. 12 reports a *priority-safe probe* that restores full node coverage without ever trading the top two priorities.

We judge learning against the priority-optimal placement π_t^* computed on *true* outcomes over the same candidate menu (Prop. 1) — not against an unattainable global optimum, and not against a scalarised score. Physical outcomes are reported as primary throughout. The scalar ε -lexicographic aggregate is used only as a convergence check (Sect. 6).

6 Theoretical Analysis

We prove four results: thresholded-lexicographic optimality, scale-invariance, a tracking bound with a conditional rate, and linear-time decisions. Objectives are normalised to $[0, 1]^4$ (Sect. 3.2), with $w_k = \beta^{-k}$ and $\|w\|_1 = \sum_k \beta^{-k}$. The proofs are short and elementary and are given in full in Sect. 7. None of these results refers to what the coordinates *mean*. They hold for any ordered bundle of bounded objectives.

Proposition 1 (Thresholded-lexicographic optimality). *On true objectives, Algorithm 1 with zero slack ($\eta = \mathbf{0}$) and no active stability hold returns a strict lexicographic optimum of $\mathcal{C}$: it minimises objective 0, breaks ties by objective 1, then 2, then 3, by the selection loop itself with no scalarisation. When the stability rule retains the incumbent, the guarantee weakens to lexicographic-optimality on objectives 0–2, cost-optimal up to that hold. With slack η , every returned $\hat{\pi}$ is within η_k of the per-objective optimum on objective k for $k \leq 2$ (a thresholded/ η -lexicographic optimum).*

The geometric scalar $L(\mathbf{g}) = \sum_k w_k \mathbf{g}_k$ is only a bounded aggregate for the convergence plot: it realises the order down to a *granularity* $((\sum_{j>k} w_j R_j)/w_k \approx 0.14$ at $\beta=8$), an ε -lexicographic surrogate, while the *controller* itself stays strict (Prop. 1).

Proposition 2 (Scale-invariance). *Let $\phi = (\phi_0, \dots, \phi_3)$ be any strictly increasing per-objective reparameterisations, and replace each objective $\mathbf{g}_k$ by $\phi_k \mathbf{g}_k$ in the estimates the selector sees. Strict lexicographic selection (Alg. 1, $\eta = \mathbf{0}$) returns the same placement. A fixed-weight scalariser is not invariant in general: there exist candidate sets and positive coordinate rescalings that change its selected optimum.*

Scale-invariance under strictly increasing reparameterisation is classical [18,15]. Our point is its *operator consequence*: a fixed-weight sum encodes a substitution rate in the objectives’ numeric units. Mis-estimating a scale silently re-prioritises, whereas the ranking does not. Note that the proposition indicts *any* method that prices the objectives rather than only the linear sum: a constrained formulation that enforces the top priority as a hard constraint and then scalarises the remainder is scale-invariant on the constrained coordinate and scale-*dependent* on all

the others, which Sect. 9.3 measures. For two candidates A, B , write $d_k = A_k - B_k$ and $R = \sum_{j \neq k} w_j (A_j - B_j)$. Scaling coordinate k crosses their weighted ordering at $c = -R/(w_k d_k)$ only when d_k and R have opposite signs. Other candidates may still prevent this pairwise crossing from changing the global argmin. The $[0, 1]$ clamp of Sect. 3.2 is strictly increasing only below saturation. This invariance covers a divisor or ρ change while no candidate is clamped, saturation can create ties and change the choice. The RQ3 sweep (Sect. 9.3) multiplies the selector’s already-clamped estimates by c , a strictly increasing map. It lies inside the proposition’s premise. A rank-normalising scalariser is also invariant but must re-estimate that ordering as ranges drift (a quantile-lex surrogate *replicates* LEXI, Sect. 12).

Counterfactual identification. The causal outcome model (Def. 4) identifies $\text{do}(\pi)$ under one explicit structural assumption.

Assumption 1 (Additive no-interference / stable unit treatment value assumption (SUTVA)). *Each objective decomposes as a sum (latency: critical path) of per-(stage, node) terms that depend on the stage’s assignment and the exogenous request state rather than on which other stages are co-placed. Component observations from an executed placement update the corresponding per-node models. Counterfactual prediction for other placements additionally requires model adequacy and coverage of their node-feature combinations.*

Assumption 1 is the paper’s principal structural limitation, and it deserves more than a sentence. We set out exactly what it buys, exactly how it fails, and what the failure costs.

Identification benefit. Additivity is the reason observations can transfer across placements. If a candidate’s objective were an arbitrary function of the whole assignment vector, then observing the outcome of one placement would say nothing about the other $|\mathcal{C}| - 1$, and the only way to learn would be to try them — which in a continuum means a real migration with a real cost. Additivity converts one execution into K per-(stage, node) observations that recombine into an estimate for every candidate sharing those nodes. It does not identify an unvisited node or an unexcited feature direction. Every off-policy claim also depends on the coverage conditions below.

Failure modes. The clearest violation is *co-location contention*. When several stages of the same placement land on one node, they queue behind each other, ensuring that node’s per-stage latency depends on how many *other* stages the placement put there — a function of the assignment vector rather than of the stage’s own assignment. Critical-path latency then stops being a sum of per-(stage, node) terms. Other violations of the same shape exist: cache and connection-pool warming between adjacent stages (which makes co-location *helpful* rather than harmful), cross-tenant interference from workloads outside the DAG, and network-path sharing between concurrent stages.

Effect of misspecification. The failure is not specific to LEXI: it degrades *every* method whose world-model is additive, which in our panel is all of the learned ones — the weighted sums, the scale-robust scalarisers, and the constrained baseline included, since all of them consume the same estimates. Sect. 9.4 measures this directly by sweeping the contention coefficient, and finds a graded degradation rather than a cliff, with the method ordering preserved except at the most extreme contention level. It also shows damage to both the top priority and privacy: SLO violations rise because predictions omit the interaction, and exposure rises as private placements become infeasible. The selector still applies the stated order to misspecified estimates. The end-to-end outcome is nevertheless degraded.

Partial mitigation. A contention-aware extension adds a learned $M/G/1$ -flavoured queueing term keyed on the number of co-placed stages, which is still estimable from executed placements. It recovers 57% of the light-contention SLO loss (Sect. 12), and less as contention grows. We report it as a partial repair rather than a solution: the assumption is genuinely restrictive, and a deployment with heavy shared-node interference is outside what this paper validates.

Theorem 1 (Top-priority tracking, under assumptions). *Let π_t^* be the priority-optimal placement (Prop. 1) in the time-varying menu $\mathcal{C}_t$, and let $\varepsilon_t = \max_{\pi \in \mathcal{C}_{t,k}} |\hat{\mathbf{g}}_{t,k}(\pi) - \mathbf{g}_{t,k}(\pi)|$. Then the top-priority (normalised SLO-hinge) objective is tracked within the slack and estimation error:*

$$\mathbf{g}_0(\hat{\pi}_t) \leq \mathbf{g}_0(\pi_t^*) + \eta_0 + 2\varepsilon_t, \quad \sum_{t \leq T} [\mathbf{g}_0(\hat{\pi}_t) - \mathbf{g}_0(\pi_t^*)] \leq \eta_0 T + 2 \sum_{t \leq T} \varepsilon_t.$$

Suppose further that (A1) each per-node estimator’s target is linear in its features up to a per-term approximation error, which induces a uniform prediction floor ε_{mis} ($\varepsilon_{\text{mis}} = 0$ under exact realizability), (A2) features and parameters are bounded, $\|x\|_2 \leq L$ and $\|\theta^*\|_2 \leq B$, and observation noise is conditionally R -sub-Gaussian, and (A3) persistent excitation: the regularised Gram matrix $A_{m,t}$ of every per-node estimator queried by a candidate in $\mathcal{C}_t$ satisfies $\lambda_{\min}(A_{m,t}) \geq ct$ for all $t \geq t_0$, for some $c > 0$. Then, with probability at least $1 - \delta$, the online ridge estimators [1] give $\sum_{t \leq T} \varepsilon_t = O(\sqrt{dT \log(MT/\delta)}) + \varepsilon_{\text{mis}}T$ for feature dimension d and M nodes, with constants depending on K, L, B, R, c, t_0 . The cumulative top-priority gap is $\eta_0 T + O(\sqrt{T \log T}) + 2\varepsilon_{\text{mis}}T$. The realised aggregate also tracks its estimate, $|L(\mathbf{g}(\hat{\pi}_t)) - L(\hat{\mathbf{g}}(\hat{\pi}_t))| \leq \|w\|_1 \varepsilon_t$. By Prop. 1 the controller is η -thresholded-lexicographic on the estimates up to $\|w\|_1 \varepsilon_t$.

We bound the normalised hinge loss rather than raw latency or the number or probability of SLO violations. Translating it into a violation-rate bound would require a separation assumption. We bound the *top priority* rather than scalar regret, since the ε -lexicographic aggregate can go negative. The per-step inequality needs no statistical assumption. The rate does. Assumption (A3) is strong for a no-exploration policy: an unvisited node keeps its prior, and visiting a

node often is not enough, because repeated collinear observations (e.g. a node executed only for requests from its own region, where the same-region feature is constant) leave some feature directions unidentified. The $O(\sqrt{T \log T})$ term is therefore *conditional*. The priority-safe probe (Sect. 12) restores node visitation, a necessary but not sufficient condition for (A3). This is estimation tracking. The separation from a well-tuned scalariser comes from Prop. 2.

Proposition 3 (Efficient decisions). *One control step costs $O(|\mathcal{C}|K)$.*

We treat stability (hysteresis) *empirically*, because churn is not provably monotone in η , and Sect. 9.6 measures a monotone churn/SLO trade-off. Counterfactual prediction avoids trying placements. LEXI decides in tens of microseconds.

7 Proofs

This section gives the full proofs of the four results stated in Sect. 6. Notation, objectives, and the weights $w_k = \beta^{-k}$ are as defined there.

Prop. 1 (thresholded-lexicographic optimality).

Proof. With $\eta = \mathbf{0}$, iteration k restricts the surviving set I to $\{\pi \in I : \mathbf{g}_k(\pi) = \min_I \mathbf{g}_k\}$. After $k = 0, 1, 2$ and the cost tie-break, $\hat{\pi}$ is minimal in the lexicographic order by construction, provided the stability rule retains no incumbent. When it does, it may keep an incumbent that is lexicographic-optimal on objectives 0–2 but not cost-minimal, giving the weaker guarantee stated. With slack, iteration k keeps $\{\pi \in I : \mathbf{g}_k(\pi) \leq \min_I \mathbf{g}_k + \eta_k\}$. $\hat{\pi}$ exceeds the surviving per-objective optimum by at most η_k . $\square$

Prop. 2 (scale-invariance).

Proof. Strict lexicographic selection depends only on the *orderings* induced by each $\mathbf{g}_k$ (argmins and tie sets per level). A strictly increasing ϕ_k preserves the order of objective k 's values, hence every per-level argmin and tie set. The selected placement is unchanged. For a weighted sum and two candidates A, B , put $d_k = A_k - B_k$ and $R = \sum_{j \neq k} w_j (A_j - B_j)$. After scaling coordinate k by $c > 0$, their score difference is $cw_k d_k + R$, which changes sign at $c = -R/(w_k d_k)$ exactly when d_k and R have opposite signs. Such candidate sets exist, establishing non-invariance, with more candidates, a pairwise crossing changes the global argmin only when no third candidate remains better. $\square$

Thm. 1 (top-priority tracking).

Proof. $\hat{\pi}_t$ lies in the η_0 -band. $\hat{\mathbf{g}}_0(\hat{\pi}_t) \leq \hat{\mathbf{g}}_0(\pi_t^*) + \eta_0$, with $|\hat{\mathbf{g}}_0 - \mathbf{g}_0| \leq \varepsilon_t$ uniformly on $\mathcal{C}_t$, $\mathbf{g}_0(\hat{\pi}_t) \leq \mathbf{g}_0(\pi_t^*) + \eta_0 + 2\varepsilon_t$, summed over t . The aggregate tracking is the sandwich $|L(\mathbf{g}) - L(\hat{\mathbf{g}})| \leq \|w\|_1 \varepsilon_t$ at $\hat{\pi}_t$. These two statements are deterministic.

For the rate, fix one per-node ridge estimator with Gram matrix $A_{m,t}$ (the ridge term plus the fixed pseudo-observation prior plus $\sum_{i < t} x_i x_i^\top$) and write its targets as $y_i = x_i^\top \theta^* + b_i + \xi_i$, with approximation error b_i (A1) and conditionally

R -sub-Gaussian noise ξ_i (A2). By the self-normalised bound of Abbasi-Yadkori, Pál, and Szepesvári (2011, Thms. 1–2), with probability at least $1 - \delta'$, simultaneously for all t , the estimation error e_t that remains after removing the approximation term below satisfies $\|e_t\|_{A_{m,t}} \leq \beta_t = R\sqrt{d \log((1 + tL^2/\lambda)/\delta')} + \sqrt{\lambda} B + b_0$, where b_0 is a constant bounding the prior’s residuals. For any feature vector x queried at step t , Cauchy-Schwarz in the $A_{m,t}$ -norm and (A3) give $|x^\top e_t| \leq \|x\|_{A_{m,t}^{-1}} \beta_t \leq L\beta_t/\sqrt{c}t$ for $t \geq t_0$. The approximation part is $|x^\top A_{m,t}^{-1} \sum_i x_i b_i| \leq \|x\|_{A_{m,t}^{-1}} \|b\|_2 \leq (L/\sqrt{c}) \max_i |b_i|$, a constant floor. A union bound over the $3M$ estimators (latency and carbon ridge, cost running mean, per node, $\delta' = \delta/(3M)$) makes these bounds hold jointly. A candidate’s estimate aggregates at most K per-(stage, node) terms (sums, or a maximum over paths of sums for latency, which is 1-Lipschitz in the sup-norm), and the hinge, the divisors and the clamp are Lipschitz with constants at most $1/\rho$ and the inverse divisors. Hence, for $t \geq t_0$, $\varepsilon_t \leq C_1 \sqrt{d \log(MT/\delta)}/t + \varepsilon_{\text{mis}}$, where C_1 depends on K, L, B, R, c and the normalisation constants and ε_{mis} collects the floors. Since every normalised coordinate lies in $[0, 1]$, $\varepsilon_t \leq 1$ for $t < t_0$. Summing, with $\sum_{t \leq T} t^{-1/2} \leq 2\sqrt{T}$, $\sum_{t \leq T} \varepsilon_t \leq t_0 + 2C_1 \sqrt{dT \log(MT/\delta)} + \varepsilon_{\text{mis}}T = O(\sqrt{T \log T}) + \varepsilon_{\text{mis}}T$ for fixed d, M, δ . Substituting into the summed per-step inequality gives the stated cumulative gap. Without (A3) the argument fails: the self-normalised bound controls the error only along directions the data have excited, and counting node updates does not ensure every direction is excited. $\square$

Prop. 3 (efficient decisions).

Proof. PREDICT evaluates K per-node estimators for each of $|\mathcal{C}|$ candidates and critical-paths in $O(K)$, the bands and tie-break are $O(|\mathcal{C}|)$, and UPDATE touches the K executed estimators. $\square$

8 Experimental Setup

We ask five core questions (RQ1–RQ5), plus two stress extensions (RQ6/RQ7). **RQ1 (Tuning)**: can a weighted sum satisfy the order, and at what tuning cost? **RQ2 (Satisfaction)**: how does LEXI, with *no* weights, compare to a hand-calibrated sum, a constrained baseline, and *scale-robust* scalarisers (rank, min-max, augmented Tchebycheff)? **RQ3 (Scale-invariance)**: do methods keep the order when a scale is misjudged? **RQ4 (Robustness)**: under candidate curation, the SLO threshold, and non-additivity? **RQ5 (Cost & slack)**: the stability/slack trade-off, a unit-free slack, and decision latency. **RQ6/RQ7** cover fourteen model- and deployment-stress extensions (Sect. 12).

Simulator and protocol. The evaluation runs on a purpose-built discrete-event simulator of the continuum, released with the artifact. The continuum has $M=7$ nodes over 3 regions (3 edge, 2 cloud, 2 serverless). The DAG has $K=6$ stages with a 260 ms SLO. A geo-diurnal workload varies load and origin, and carbon follows a synthetic diurnal signal with seeded noise. Each decision ranks a fixed

40-candidate set. All methods share it, and RQ4 stresses this with a random set. Over a 1,600-step horizon, the primary comparison (RQ2) averages **15 seed-level summaries** with **95% bootstrap confidence intervals (CIs)** over the post-warm-up window (steps 800–1599). Sweeps and scaling use 6–8 seeds, as noted per result. Paired-bootstrap tests resample those summaries.

8.1 How the evaluation inputs are generated, and why

Node energy, per-invocation cost, queueing sensitivity, and the assignment of PII tags to particular stages are all authorial choices, and the published DeathStar-Bench [20] and Alibaba microservice traces supply DAG *shapes* but neither PII tags nor a continuum placement. This subsection states every choice, its value, and the reasoning behind it. The governing principle throughout was **to create genuine objective tension**: if any single tier were best on all four objectives, the placement problem would be trivial and no decision rule could be distinguished from another. We name this bias explicitly because it is the strongest threat to the construct validity of the whole evaluation, and return to it at the end of the subsection.

The fleet. Table 2 gives the seven nodes verbatim. The design is a three-way trade with no dominating tier:

- **Edge** is the only *private* tier and has low RTT *when the request origin is in its region* (4–5 ms) but a punitive inter-region RTT (70–72 ms). It is compute-weak (≈ 2.1 ms per unit of stage demand, $4\times$ cloud), the most expensive per invocation (1.10–1.15 simulator units), the most queueing-sensitive ($q \approx 8.5$ –9.0 ms per unit load), and has the highest synthetic effective intensity (430–540 gCO₂/kWh). Keeping a PII stage private therefore costs latency, carbon, and money — which is what makes privacy a real trade-off rather than a free lunch. The carbon-dirty edge is the least obvious choice. It reflects small on-prem sites drawing from a regional rather than a sited-for-renewables grid, and it is what prevents privacy and carbon from being trivially aligned.
- **Cloud** is compute-strong (0.50–0.55) and cheap-ish (\$0.50–\$0.55) but pays 35–50 ms RTT from anywhere and is public. Its energy per unit demand is the *highest* (0.90–0.95 Wh), reflecting general-purpose instances kept warm.
- **Serverless** is the cheapest (\$0.18–\$0.20), the most energy-efficient (0.34–0.36 Wh, reflecting scale-to-zero), and sits on the cleanest grids (175–260), with middling compute and RTT — and it is public. It is deliberately the tempting option: a scalariser that prices privacy too low will reach for it, which is exactly what happens in Sect. 4.

Absolute magnitudes are in simulator units and are not claimed to reproduce any specific provider’s price list or power draw. The ordering and ratios between tiers determine the simulated trade-offs, and follow the qualitative consensus in the continuum literature (low-latency-but-weak edge [5,35], powerful-but-remote cloud [38], cheap-and-elastic serverless [22]).

Table 2. The seven continuum nodes of config A, verbatim from the artifact (edge_A/B/C, cloud_A/C, srv1_B/C). Edge nodes are private.

Id	Tier	Reg.	Latency terms				Cost and carbon			
			b	RTT _{in}	RTT _{out}	q	Cost	Energy	$\bar{I}$	Priv.
			ms/unit	ms	ms	ms/load	sim./inv.	Wh, $d=20$	g/kWh	
E0	edge	0	2.20	4	70	9.0	1.15	0.62	520	✓
E1		1	2.00	5	72	8.5	1.10	0.58	540	✓
E2		2	2.10	5	71	8.8	1.12	0.60	430	✓
C0	cloud	0	0.50	35	46	2.0	0.55	0.95	360	
C2		2	0.55	38	50	2.2	0.50	0.90	190	
S1	serverless	1	1.10	18	30	4.5	0.20	0.36	260	
S2		2	1.20	20	32	4.8	0.18	0.34	175	

The service DAG and the PII tags. The evaluated DAG is the six-stage pipeline `frontend(8) → auth(16) → {search(30), recommend(25)} → order(20) → pay(18)`, with per-stage compute demand in parentheses. The branching shape — a serial head, a compute-heavy parallel fan-out, a join, and a serial tail — follows the social-network and e-commerce request graphs of DeathStarBench [20] and the Alibaba microservice traces [26]. The demands are set so the parallel branches dominate the critical path. The `auth` and `pay` stages nevertheless contribute latency while also determining privacy. The two objectives are coupled rather than assigned to disjoint stages.

The PII tags on `auth` and `pay` are ours rather than the traces’. Those two stages handle credentials and payment tokens, which is the conventional reading of such a pipeline, but nothing in the published traces asserts it. The relevant design property concerns the tag distribution: (i) exactly two of six stages carry it. The privacy coordinate is integer-valued in $\{0, 1, 2\}$ and genuinely tie-prone, and (ii) the tagged stages are *not* the critical-path-dominating ones. Privacy and latency pull in different directions. Config B varies this independently: 10 nodes, an 8-stage DAG, and *three* PII stages, with a different economic structure (cheaper and greener edge, dirtier cloud). The method ordering holds there too (Sect. 9.4), which is the only evidence we have that the conclusions are not an artefact of the config-A tagging.

The physical sub-models. Per-(stage, node) latency is $\text{RTT}(\text{origin}, n) + b_n \cdot \text{demand}_s + q_n \cdot \ell$ (RTT_{in} within the node’s region, RTT_{out} across, b : compute per unit demand, q : queueing sensitivity per unit load, b and energy are per unit demand), and end-to-end latency is the critical path over the DAG. Carbon is $\sum_s \text{energy}_n \cdot (\text{demand}_s/20) \cdot I_n(t)/1000$ grams with $I_n(t) = \bar{I}_n + A_n \sin(2\pi(t \bmod 240)/240 + \phi_n)$, i.e. a node-specific synthetic effective intensity $\bar{I}_n$ (gCO₂/kWh) modulated by a 240-step diurnal cycle with a per-node phase and an amplitude A_n of 60–120 gCO₂/kWh. Cost is $\sum_s \text{cost}_{\pi(s)}$. Execution adds seeded noise: queueing noise $\mathcal{N}(0, 0.10(1 + q_n \ell))$ per stage, truncated at zero, and multiplicative carbon noise $\mathcal{N}(1, 0.03)$.

The queueing-sensitivity values (q) are the parameter with the least external grounding. We chose them proportional to inverse compute capacity (weak edge queues hardest) and large enough that load genuinely re-orders the tiers over a day, since a queueing term that never bites would make the online latency model pointless. Sect. 9.4’s SLO sweep (220–340 ms) is the sensitivity analysis for this choice: it moves the operating point from 59% to 0% violations and the method ordering survives.

The workload. Load follows a diurnal sinusoid with $\mathcal{N}(0, 0.10)$ noise, floored at 0.1. The request origin region is drawn from time-of-day-dependent weights peaking at 0.20, 0.50 and 0.80 of the day for regions 0, 1, 2. Which node is latency-optimal drifts continuously and a *fixed* placement is repeatedly wrong-footed. This is what the static heuristics in Table 3 are being tested against, and it is why they do badly.

The candidate menu — the most consequential choice. Each decision ranks a fixed menu of $|\mathcal{C}| = 40$ placements, built once per configuration and shared by every method. It is 22 *structured* placements plus 18 seeded random fills, deduplicated. The structured part is: the 7 homogeneous placements (all stages on one node), 3 latency-greedy placements, one per origin region, assigning each stage its nominal-latency-minimal node, 12 privacy-aware placements enumerating every (private edge, public backend) pair, ensuring that for *any* origin there exists a region-matched private option that can still meet the SLO, and the cost-minimal placement, which coincides with one homogeneous placement. The enumeration of all 3×4 edge/backend pairs is the important one — it guarantees the privacy objective always has something to choose. A privacy failure is a failure of the *rule* rather than of the menu.

This is a curated menu, and we do not present it as neutral: Sect. 9.4 shows it is the single largest determinant of the absolute numbers in this paper, and analyses exactly why. A deployable alternative — a greedy per-objective planner that constructs candidates online rather than enumerating them — recovers 99% of the curated menu’s quality (Sect. 12), which is our evidence that the curation is reproducible by a real system and not an oracle in disguise.

What was tuned, and by whom. LEXI has no weights to tune. The parameters above were fixed *before* the comparison and were not adjusted to favour any method. Where a baseline has weights, we tuned them *in the baseline’s favour*: WS-tuned is a calibrated, SLO-heavy vector that is better than the best point of the 84-point objective-simplex sweep of RQ1 (a finer 220-point sweep is in Sect. 12), and the scale-robust scalarisers reuse that same tuned vector so any gap is attributable to the *aggregation* rather than to the weights. The one parameter chosen with knowledge of the outcome is the SLO itself (260 ms), picked so that the nominal operating point is neither trivially satisfiable nor infeasible. The sweep in Sect. 9.4 reports what happens at 220, 300, and 340 ms.

Summary of the construct threat. The simulator was built to exhibit priority conflict, and it does. A reader should therefore treat the *absolute* numbers (“1.46% SLO violations”) as properties of this configuration rather than as forecasts, and weigh instead the *relative* and *structural* findings: that the tuned-weight region is small, that a ranking is invariant where a pricing is not, and that the rule attains the optimum of whatever menu it is given. Those are the claims that survive a change of parameters, and Sect. 9.4 tests each on an independent configuration.

8.2 Real-trace grounding

For external validity we also drive two signals from real public traces, *without re-tuning*: regional forecasts from the UK Carbon Intensity API [30] for North Scotland, London, and Yorkshire over 1–15 January 2024 at half-hourly resolution, and request load from the Azure Functions 2019 trace [34] (day-one, per-minute arrivals). Sect. 11 re-runs the full primary comparison under these traces. Node energy, cost, RTT, compute capacity, queueing sensitivity, the DAG, and the PII tags remain synthetic. We are explicit that real traces ground *two* of the simulator’s inputs and not the fleet.

8.3 Methods and metrics

LEXI is the strict lexicographic controller ($\eta=0$), LEXI(η) adds default slack (RQ5) and LEXI-*auto* sets it from each objective’s candidate interquartile spread. Baselines share the candidate set, model architecture, and initial priors. Each learned method maintains a separate model updated from its own selected placements. Estimates can diverge: *WS-default* (untuned weights 0.30, 0.12, 0.30, 0.28), *WS-tuned* (calibrated, exponentially-separated, SLO-heavy: 0.85, 0.10, 0.00, 0.05, RQ1 also sweeps the 84-vector simplex, best on-grid worse still), three *scale-robust* scalarisers reusing WS-tuned’s weights (*Rank-WS*, *Norm-WS*, *Tcheby* [29,37]. Any gap is the *aggregation*), *Lex-LP* (exact discrete lexicographic), *Constr-WS*, a one-step SLO-constrained selector (admits only candidates predicted SLO-feasible, then minimises an equal-weight (0.34, 0.33, 0.33) privacy/carbon/cost sum, if none is predicted feasible, it chooses the candidate with minimum predicted SLO hinge. It learns no policy, value function or Lagrange multiplier, and represents the decision rule shared by constraint-first formulations [3]), *SO-RL* (latency-only tabular RL), and *static-greedy/binpack* heuristics. Physical outcomes are *primary*: SLO-violation (%), PII exposure (of 2), carbon (g/req), cost (\$/req), churn (stages/req), each with a 95% bootstrap CI over 15 seeds and paired-bootstrap headline tests. We also report the *priority-inversion rate*: the fraction of decisions that are lexicographically worse than the strict priority optimum computed on the same shared estimates — a rule-level diagnostic that is independent of the outcome scale. Scalar regret is only a secondary ε -lexicographic surrogate (Sect. 6) and is not a fair cross-method headline.

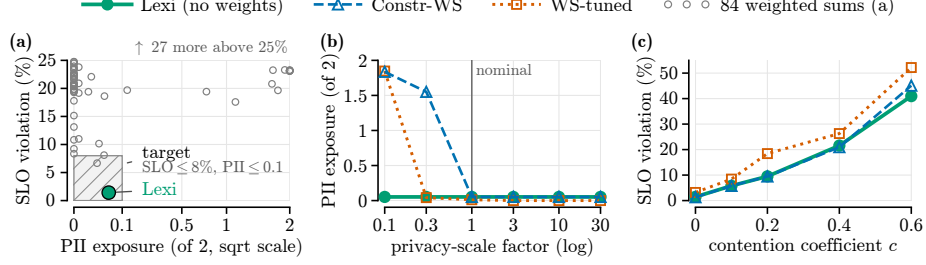


Fig. 2. (a) RQ1 weight sweep, hatched: target (6 seeds). (b) RQ3 privacy-scale miscalibration (8 seeds). (c) RQ4 co-location contention, breaking Assumption 1 (15 seeds).

9 Results

The primary comparison (RQ2) averages 15 seed-level summaries with 95% bootstrap CIs over steps 800–1599. The sweeps, scale-invariance, and scaling experiments use fewer seeds (6–8), as noted at each result. Every number below is regenerated by a single command from the released artifact.

9.1 RQ1: weight sensitivity

We swept 84 weight vectors over the objective simplex (Fig. 2(a), a finer 220-vector sweep appears in Sect. 12). Only **one** of 84 meets a modest priority target ($\text{SLO} \leq 8\%$ and $\text{PII} \leq 0.1$), and **none** dominate LEXI. LEXI uses the order alone and lands on the frontier at SLO 1.42%/PII 0.05 in this 6-seed sweep (1.46% in the 15-seed main run of Table 3), while the untuned WS-default sits far off at 23%/1.94. A weighted sum can therefore reach the frontier, but only from a small region found by a deployment-specific search.

9.2 RQ2: objective satisfaction and priority inversions

Table 3 reports the operator-facing outcomes: LEXI uses no weights, WS-tuned is calibrated, and Constr-WS constrains the SLO, the priority-inversion rate $\text{Inv}\%$ counts decisions that trade a higher priority for a lower one, and is undefined for fixed heuristics, which do not rank the estimates. LEXI attains SLO-violation **1.46%** (95% CI [1.31, 1.60]) and PII exposure **0.05**, on par with the SLO-constrained sum Constr-WS (1.32%/0.05) and *better on SLO* than the hand-tuned weighted sum (3.19%, paired $p < 0.001$), which is in turn better on privacy (0.01). The three occupy a nearby target region with different residual trade-offs. Only LEXI uses the order alone. (A lexicographic-LP optimiser is byte-identical to LEXI here, agreement 1.00 over 24k decisions.) The scale-robust sums reach near-LEXI privacy, but at 2.3–3.3 $\times$ its SLO violation, because they still price the objectives. The untuned sum and the heuristics are far worse. Only LEXI is *priority-exact*: it never inverts the order (0.0%), whereas the constrained and scale-robust baselines invert on 0.7 to 5.0% of decisions and the untuned sum on 94.7%.

Table 3. RQ2 operator-facing outcomes (15 seeds, means), lower is better. Inv% is the priority-inversion rate (n/a for fixed heuristics).

Method	SLO%↓	PII↓	Carbon↓	Cost↓	Churn↓	Inv%↓
LEXI (no weights)	1.46	0.05	0.98	3.21	1.35	0.0
Constr-WS (SLO filter + sum)	1.32	0.05	0.98	3.19	1.32	0.7
WS-tuned (calibrated)	3.19	0.01	0.99	3.20	1.28	3.7
Rank-WS (scale-robust)	3.48	0.00	0.99	3.22	1.29	4.2
Norm-WS (scale-robust)	3.30	0.01	0.98	3.20	1.29	3.6
Tchebycheff (scale-robust)	4.87	0.01	0.97	3.20	1.28	5.0
WS-default (untuned)	23.02	1.94	0.45	1.22	0.65	94.7
SO-RL (latency-only)	11.53	1.84	1.22	2.70	1.99	n/a
static-greedy / binpack	67.2/67.8	2.00	1.95/ 0.36	3.90/ 1.08	0.00	n/a

Operational conflict. We stress the comparison with a cold start, a common serverless effect that makes the cheap, green option risky for latency. As the cold-start penalty grows, every priced method inverts the order on **22 to 37%** of decisions, up from under 5%. This includes the scale-robust rank- and min-max-normalised sums and the augmented Tchebycheff, where a large low-priority gain outvotes a small top-priority loss. LEXI stays priority-exact (**0%**) throughout, and its margin variant, at a 30 ms cold-start, holds realized SLO violation to **22%** where the tuned sum reaches 34%. The full breakdown is Sect. 10, and the re-run under real carbon and load traces is Sect. 11. Both preserve every RQ2 conclusion, and the real bursty load makes the priority-inversion gap *sharper* (16–17% for every priced scalariser, 14% for the constrained baseline, 0% for LEXI).

9.3 RQ3: sensitivity to objective scaling

Rescaling the *units* of the privacy objective as the optimiser sees them—leaving the true outcome unchanged (Fig. 2(b))—holds LEXI’s PII exposure **invariant** at 0.052 across a 300× swing, because it depends only on the objective *order* (Prop. 2). The tuned weighted sum matched LEXI at nominal scale, but it silently re-prioritises: under-weighting privacy 10× drives its PII from 0.01 to **1.85 of 2 stages**, and the augmented Tchebycheff to 1.86. Such a scale drifts and cannot be measured exactly, and that is what breaks the tuned order but not the lexicographic one. Scale invariance alone is insufficient: the rank- and min-max-normalised sums are also invariant, yet they sit at a *worse* point (SLO ≈ 3.5% vs. 1.46%) because they still combine the ranked objectives numerically. Strict lexicographic control is both scale-invariant and priority-exact in this test.

Comparison with Constr-WS. Constr-WS also requires no tuning for its SLO constraint and performs comparably to LEXI on the headline row (1.32% vs 1.46% SLO, with identical PII and carbon, and cost within 0.02).

Agreement on the top priority. Constr-WS admits only the candidates predicted to meet the SLO, then minimises an equal-weight sum of privacy, carbon, and

Table 4. RQ3 privacy-scale perturbation with Constr-WS added (8 seeds): PII exposure (of 2) when the optimiser’s view of privacy is scaled by the factor. True outcomes unchanged.

Privacy scale factor	0.1	0.3	1.0	3.0	10.0	30.0
LEXI (strict)	0.052	0.052	0.052	0.052	0.052	0.052
Constr-WS	1.837	1.551	0.052	0.052	0.052	0.052
WS-tuned	1.848	0.042	0.012	0.000	0.000	0.000
Tchebycheff	1.855	0.312	0.007	0.000	0.000	0.000

cost over them. When this feasible set is nonempty, it equals level 0 of the strict lexicographic rule. If the set is empty, Constr-WS instead chooses minimum predicted hinge. Hysteresis and separate learned-model trajectories also affect the realized results. The nearby headline values do not establish algorithmic equivalence.

Differences below the constraint. Constr-WS has one priority level and then a price. Lexi has four levels and no price. Three measured consequences follow.

1. *Priority exactness.* Constr-WS inverts the order on 0.68% of decisions (95% CI [0.55, 0.83]) in the benign synthetic regime and on **14.3%** under the real Azure load (Sect. 11), where conflict among the lower three objectives is frequent. LEXI is at exactly 0% in both. Every one of those inversions is a decision that traded privacy for carbon or cost, which the stated order forbids.
2. *Scale-invariance stops at the constraint.* Prop. 2 protects only the coordinates a method *ranks*. We re-ran the RQ3 privacy-rescaling sweep with Constr-WS added to the panel (Table 4 and Fig. 2(b), 8 seeds). Under a 10× privacy under-scaling its PII exposure rises from 0.052 to **1.84 of 2 stages** — statistically the same failure as the tuned weighted sum’s 1.85. The hard SLO constraint is scale-free. The equal-weight blend underneath it is not. LEXI stays exactly invariant across the whole sweep.
3. *Generality of the frontier.* On the independent config B (10 nodes, 8 stages, three PII stages) LEXI reaches 6.78% SLO against Constr-WS’s 7.17%, with both at zero PII (Sect. 9.4).

Constr-WS is competitive at the nominal point and needs no tuning for its SLO constraint. Its priority correctness applies to that constrained objective, while the objectives below it remain combined by a weighted sum. The inversion rate and scale sweep quantify this distinction. For an operator whose specification has only one level—meet the SLO before an otherwise scalar objective—Constr-WS is the simpler suitable method.

9.4 RQ4: robustness to curation, configuration and additivity

Candidate-menu sensitivity. Replacing the curated 40-candidate menu with a uniformly random one moves LEXI from 1.44% to **43.6%** SLO violations (8

seeds). The following analysis separates selection error from the feasible outcomes made available by each menu.

Menu composition. Table 5 reports the anatomy of the two menus, measured over the evaluation window: the share of the 40 candidates that meet the SLO, keep both PII stages private (PII-clean), or do both at a typical request, the oracle and LEXI outcomes, and LEXI’s mean top-priority gap to the oracle in normalised units. In the curated menu, 13.7% of the 40 candidates meet the SLO at a typical request and 45.0% keep both PII stages private, with 5.6% (about two candidates) doing both — the rule therefore almost always has a feasible, private option to find. In the random menu those figures collapse to 3.9%, 19.4% and 0.2%: at a typical request, fewer than one candidate in four hundred is both SLO-feasible and PII-clean. The 19.4% figure is not mysterious — placing six stages uniformly over seven nodes keeps both PII stages on one of the three edge nodes with probability $(3/7)^2 = 18.4\%$ — and the 3.9% follows from the fact that a random assignment routinely scatters critical-path stages onto inter-region nodes at 70 ms RTT apiece.

Menu oracle. The *expected-objective oracle* selects the per-step priority-optimal candidate from noise-free simulator expectations, then evaluates that placement under a realized outcome. It is not an upper bound on every finite realized sample. On the curated menu the oracle violates the SLO on 1.48% of requests. On the random menu it violates on **43.53%**. LEXI attains 1.44% and 43.56% respectively. In other words, **the jump from 2% to 44% is almost exactly the jump in what the menu makes achievable**, and LEXI’s gap to the achievable optimum is 0.0 and 3×10^{-5} in normalized top-priority units. Measured as decisions, LEXI selects the expected-objective priority optimum on 99.7% of steps with the curated menu and 98.9% with the random one.

Interpretation. The absolute SLO and PII values depend on the candidate generator and should be reported with the menu specification. RQ4 also establishes that the rule remains optimal-on-its-menu when the menu is destroyed, and that the *method ordering* is preserved (43.6% for LEXI against 45.8% for WS-tuned and 59.8% for WS-default). A rule whose quality depended on curation would lose its ordering too. The complementary result is in Sect. 12: a deployable greedy per-objective planner, generating candidates online with no enumeration, recovers 99% of the curated menu’s quality, indicating that an online planner can reproduce most of the curated menu’s benefit.

SLO threshold and an independent configuration. The ordering also holds when we sweep the SLO over 220–340 ms, which moves the operating point from near-saturation to trivial: at 220 ms every method is in distress (LEXI 58.9%, Constr-WS 58.9%, WS-tuned 73.8%, WS-default 88.0%). At 340 ms every ranked method reaches 0% while WS-default still exposes both PII stages. It holds on an independent continuum (config B: 10 nodes, an 8-stage DAG, *three* PII stages,

Table 5. RQ4 candidate-menu anatomy (8 seeds): % of candidates SLO-feasible, PII-clean or both, oracle and LEXI outcomes, LEXI’s gap to the oracle.

Menu	% cand. that are...			Oracle		LEXI		LEXI gap
	SLO-feas.	PII-clean	both	SLO%	PII	SLO%	PII	to oracle
Curated (40)	13.7	45.0	5.6	1.48	0.052	1.44	0.052	0.0
Random (40)	3.9	19.4	0.2	43.53	1.461	43.56	1.450	3×10^{-5}

Table 6. RQ4 additivity break (15 seeds): SLO violation (%) as contention c grows, LEXI’s PII, and the SLO loss a learned $M/G/1$ term recovers.

c	LEXI (strict)	Constr-WS	WS-tuned	LEXI PII	$M/G/1$ recovery
0.0	1.46	1.32	3.19	0.05	—
0.1	5.79	5.77	8.43	0.18	57%
0.2	9.50	9.46	18.43	0.65	19%
0.4	21.57	20.97	26.29	1.07	12%
0.6	40.97	45.09	52.18	1.49	2%

different economics), where $\text{LEXI } 6.78\%/0.00 < \text{Constr-WS } 7.17\% < \text{WS-tuned } 8.14\% < \text{WS-default } 13.81\%/2.73$.

Breaking additivity (Assumption 1). Sweeping the co-location contention coefficient c , which makes latency non-additive and so violates Assumption 1, degrades *every* additive-model method together, as Sect. 6 predicts (Table 6, Fig. 2(c)). The last column of the table is the share of LEXI’s contention-induced SLO loss recovered by a learned $M/G/1$ term. The degradation is graded rather than a cliff — $1.46 \rightarrow 5.79 \rightarrow 9.50 \rightarrow 21.57 \rightarrow 40.97\%$ for LEXI as c runs $0 \rightarrow 0.6$ — and the ordering is preserved at every level except the most extreme, where Constr-WS (45.1%) falls behind LEXI (41.0%) and the tuned sum (52.2%) behind both. Two further observations matter operationally. First, the damage lands on the *lower* priorities by design: at $c=0.6$ private placements are largely SLO-infeasible. PII exposure rises to 1.49 as the rule spends the second priority to defend the first (Sect. 12). Second, a learned $M/G/1$ -flavoured contention term recovers 57% of the SLO loss at light contention ($c=0.1$) and progressively less as c grows (19% at 0.2, 12% at 0.4, 2% at 0.6). We report this as a partial repair. A deployment with heavy shared-node interference is outside what this paper validates, for LEXI and for every baseline that shares its world-model.

9.5 Prediction accuracy of the counterfactual model

Beyond the decisions it produces, we validate the prediction model itself. All figures are measured over the *entire* 40-candidate menu at every post-warm-up step — that is, over the 39 candidates per step that were never executed, which is the counterfactual claim — on 15 seeds under LEXI’s own trajectory.

- **Latency**: mean absolute error **10.5** ms against a mean true candidate latency of 363.8 ms (2.9%). Note the denominator includes badly placed candidates that run far over the SLO.
- **SLO hinge**, the coordinate the top priority actually reads: MAE 0.0286 in normalised units, i.e. **1.7** ms of hinge error. The hinge is far better estimated than raw latency because it saturates at 0 for the feasible region, which is where the decision is made.
- **Carbon**: MAE **0.053** g against a fleet range of roughly 0.3–2.6 g.
- **Cost**: MAE **\$0.16**. Cost is deterministic given the node. This residual is entirely a *coverage* artefact — nodes the no-exploration policy never visits keep their prior. It is the empirical face of a failure of assumption (A3) in Thm. 1, and the priority-safe probe that lifts node coverage from 85.7% to 100% (Sect. 12) addresses it.
- **Privacy** is not estimated: it is read off placement metadata and is exact.
- An independent check from the doubly-robust study gives mean $|\hat{\mathbf{g}} - \mathbf{g}| = 0.0246$ on the hinge and carbon coordinates jointly, and shows the direct method is already near-unbiased under Assumption 1 (the DR correction changes no decision, Sect. 12).

Does the error change decisions? This is the question that matters, and it is answerable: LEXI selects the *true* priority optimum over its menu on **99.7%** of post-warm-up decisions (curated menu, 8 seeds) and on 98.9% with a random menu. Constr-WS, using the same model, achieves 99.1% and 87.6%. The residual error is concentrated where it is cheapest — Sect. 4 shows a typical instance, where two candidates were admitted to the SLO band in error and both were eliminated by the next priority anyway. This is what Thm. 1’s ε_t term looks like in practice.

What we still do not validate. Prediction accuracy is measured *inside the simulator*. It validates the estimator against the generative model rather than against a real system. Resource contention beyond the swept coefficient, migration cost beyond the churn count, telemetry delay, and control-plane overhead are not modelled at all (Sect. 13).

9.6 RQ5: a safety margin protects the SLO, stability is near-free

Strict LEXI meets the SLO in *expectation*, yet realised queueing noise still breaches it on 1.46% of requests. Reserving an SLO safety margin δ (Sect. 5) holds back headroom for that noise. At $\delta=4$ ms violations fall to **0.16%**, and at $\delta=8$ ms to **0.01%** (paired $p < 0.001$ against both strict LEXI and Constr-WS), spending the second priority to protect the first at a measured privacy cost (PII 0.05 $\rightarrow$ 0.11 $\rightarrow$ 0.28) — which is the stated order behaving exactly as specified. The margin helps most where model latency is underestimated: under moderate contention ($c=0.4$) it cuts SLO violations from 18.7% to 14.6%. It is one millisecond knob on a threshold objective. Rescaling the other units leaves their order unchanged (Prop. 2).

Table 7. RQ2 under conflict (15 seeds): priority inversion (%) as serverless cold-start risk grows, and realised SLO violation (%) of LEXI-margin / WS-tuned.

Cold-start (ms)	Priority inversion %↓					SLO%↓
	LEXI	WS-tuned	rank-WS	norm-WS	Tcheby	mgn/WS
0	0	4	4	4	5	0/3
15	0	23	22	23	24	9/12
30	0	28	29	27	37	22/34
45	0	24	24	24	35	32/36
60	0	24	24	24	35	32/38

Stability is near-free: a carbon slack with hysteresis cuts churn monotonically from 1.35 to 1.14 stages/req as the slack multiplier runs $0 \rightarrow 8$, while SLO violations fall slightly ($1.46 \rightarrow 0.92\%$) rather than rising, a quantile auto-slack makes the knob unit-free. Decision latency is near-linear in the candidate count (bootstrap slope 1.12×10^{-4} ms/cand, 95% CI $[1.05, 1.35] \times 10^{-4}$, 0.082 ms at 8 candidates rising to 0.096 ms at 128). A 40-candidate decision is under 0.1 ms, deployable in the request path (Prop. 3). We note the obvious caveat: this is the decision *rule*’s cost, measured in-process. It excludes everything an orchestrator must also do — reconciliation, admission, and the actual migration — none of which this paper measures.

10 RQ2 Under Priority Conflict (Cold Starts)

Sect. 9.2 reports that, under operational conflict, every priced scalariser inverts the priority order on 22 to 37% of decisions while LEXI stays exactly 0. Table 7 gives the full breakdown. We stress the priority conflict with a canonical serverless effect, the cold start, which makes the cheap, green option latency-risky. As the cold-start penalty grows, the priority-inversion rate of every priced scalariser climbs, including the scale-invariant rank- and min-max-normalised sums and the augmented Tchebycheff. LEXI stays priority-exact throughout, and its SLO safety margin holds realized SLO violation well below the tuned weighted sum.

11 Real-Trace External Validity (RQ2)

To check that the priority-first conclusions are not an artefact of synthetic parameters, we re-run the primary comparison (Table 3) with two of the simulator’s signals replaced by real, publicly available traces used by peer work, and *no method re-tuned*:

- **Carbon intensity** from real UK National Grid *regional* carbon-intensity traces (Carbon Intensity API, 14 days at half-hourly resolution, Jan. 2024), mapped to the three simulator regions to give real spatial *and* temporal variation: North Scotland (mean 92.8), London (167.5), and Yorkshire

Table 8. RQ2 outcomes under **real traces** (UK-grid carbon, Azure Functions load, 15 seeds, means, lower is better, no re-tuning). Inv% as in Table 3, n/a: not measured.

Method	SLO%↓	PII↓	Carbon↓	Cost↓	Churn↓	Inv%↓
LEXI (no weights)	2.02	0.020	0.432	3.634	1.576	0.0
Constr-WS (SLO filter + sum)	1.43	0.020	0.449	3.350	1.421	14.3
WS-tuned (calibrated)	2.22	0.002	0.453	3.355	1.345	16.3
Rank-WS (scale-robust)	1.86	0.000	0.453	3.366	1.356	16.4
Norm-WS (scale-robust)	2.43	0.002	0.453	3.351	1.344	16.4
Tcheby (scale-robust)	3.82	0.001	0.454	3.340	1.378	16.8
WS-default (untuned)	24.90	1.218	0.359	1.903	0.956	n/a

(182.9gCO₂eq/kWh). Electricity Maps publishes the same quantity per zone. We use the equivalent open API for a fully reproducible artefact. Per-node carbon then differs through node *energy* (edge less efficient than serverless), the physically correct source of intra-grid variation.

- **Request load** from the Microsoft Azure Functions 2019 invocation trace (Shahrad et al., USENIX ATC 2020, day 1, all 46,412 functions, 9.1×10^8 invocations), whose real per-minute arrival shape and burstiness drive the load time series, amplitude-matched to the simulator’s load range.
- The **service DAG** follows the shape of DeathStarBench (Gan et al., ASPLOS 2019) and Alibaba microservice traces (Luo et al., SoCC 2021).

Table 8 reports the outcome. Every RQ2 conclusion survives the move to real carbon and load. LEXI, using *no weights*, matches the hand-calibrated weighted sum on the top-priority SLO (2.02% vs 2.22%, paired bootstrap $p=0.15$, i.e. statistically on par) and the SLO-constrained sum Constr-WS (1.43%), sitting on the operating frontier at PII 0.02, while the untuned sum collapses to 24.9% SLO violation and 1.22 PII. The decisive difference is priority *exactness*: the real, bursty Azure load induces far more objective conflict than the benign synthetic regime. Every *priced* scalariser, the scale-invariant rank- and min-max-normalised sums included, inverts the strict priority order on 16–17% of decisions and the constrained baseline on 14% (up from under 5% in the synthetic benign case), whereas LEXI stays exactly 0%. Real workloads make the case for strict lexicographic control *stronger* rather than weaker.

12 RQ6/RQ7: Model and deployment stress

This section reports the full model- and deployment-stress study in support of RQ6/RQ7, which the conference version of this paper omits for space. Table 9 collects fourteen measured extensions (15 seeds unless marked, [†]2 to 5 seeds). A learned $M/G/1$ term recovers **57%** of the light-contention SLO loss. Strict LEXI is exactly invariant to the hinge scale. Decisions stay sub-millisecond to $K=30/C=1000$. A 220-weighting sweep with fully-tuned baselines leaves 0% dominating LEXI, and a hard residency pre-filter cuts residency violations to 0%. A *priority-safe probe* considers an under-visited node *only* among candidates

Table 9. Measured robustness and deployment extensions (15 seeds unless marked, [†]2 to 5 seeds).

Aspect	Extension	Measured result
Contention DR estimator	learned $M/G/1$ term carbon-only DR	57% SLO recovery (light contention) decision-safe; DM already near-unbiased ($\leq 0.5\%$)
Hinge ρ Margin δ Scaling	$8\times$ scale swing online $k\hat{\sigma}$ auto-cal. $K\leq 30, C\leq 2000^\dagger$	SLO 1.46%, PII 0.05 invariant $k=2\rightarrow\delta\approx 3.7$ ms, SLO 0.23% ≤ 0.75 ms/decision; update $O(K)$, C -indep.
Priority orders	static + dynamic	each honours top priority; reordering succeeds
Larger sweep Compliance	220 weightings [†] residency pre-filter	0% dominate; tuned baselines still price residency violations 2.7% $\rightarrow$ 0%
Safe probing Nonstation. Cand. gen. Contention $\delta\times\eta_{\text{priv}}$	LCB-gated priority-safe probe RTT/carbon drift, online update deployable greedy planner lower-priority impact 2-D margin/slack grid	cov. 85.7 $\rightarrow$ 100% , SLO 1.46 $\rightarrow$ 1.30% static 37.2% vs. online 28.9% (drift) SLO 1.9% ; recovers 99% of curated PII $\rightarrow$ 1.49 at $c=0.6$ (SLO-infeasible) δ trades SLO/PII; only slack breaks scale-inv.
Quantile	quantile-normalised lex.	100% agreement; <i>replicates</i> strict LEXI

whose mean predicted latency meets the SLO with the strict choice’s headroom and whose PII is no worse. Coverage rises **85.7%** $\rightarrow$ **100%** with the SLO *unharmful* (1.46% $\rightarrow$ 1.30%), never trading the top two priorities. A per-request ridge update tracks RTT/carbon drift. A deployable per-objective planner recovers 99% of curated quality. Heavy contention spills onto the *lower* priorities (PII to 1.49) as private placements turn SLO-infeasible. A $\delta\times$ slack grid shows δ trades SLO for privacy as the order dictates ($\delta=0/4/8$ ms $\rightarrow$ SLO 1.45/0.16/0.008%, PII 0.054/0.114/0.278), and scale-invariance degrades *only* with a nonzero privacy slack, confirming invariance is a property of *strict* selection. A controller over quantile-normalised scores **exactly replicates** strict LEXI (**100%** agreement), as Prop. 2 predicts.

13 Discussion and Threats to Validity

13.1 Scope of the contribution

At the nominal operating point, the hand-calibrated weighted sum, the SLO-constrained sum (Constr-WS), and LEXI occupy the same target region in Table 3, with Constr-WS marginally ahead on the headline SLO row (Sect. 9.3). We reach this region without weight tuning: only one of 84 swept weightings meets the priority target, and none dominate it. Its strict selection rule is also scale-invariant, whereas under-scaled privacy changes PII exposure from 0.05 to 1.85 for the tuned sum and to 1.84 for Constr-WS (Table 4).

The formal results concern the selection rule rather than a particular objective. Privacy is one instantiation of the second position, chosen because its integer values produce ties and because its units have no natural exchange rate with carbon or cost.

13.2 Model boundaries

Additivity (Assumption 1) is the principal structural limit. When stages contend for a shared node, latency stops being a sum of per-stage terms, the counterfactual identification argument fails, and every additive-model method degrades together (Table 6). We treat this at length in Sect. 6 and measure it in Sect. 9.4. The degradation is graded, the method ordering survives to $c=0.4$, the cost is pushed onto the lower priorities by design, and a learned $M/G/1$ term recovers 57% of the loss at light contention and little at heavy contention. These results establish the applicable regime: a deployment where stages of one request routinely share a saturated node requires a contention-aware model.

Scale-invariance is a property of strict selection only. The carbon slack and the SLO margin carry their own units. The quantile auto-slack removes the carbon knob’s units, and δ is a millisecond knob on a millisecond objective, but a nonzero *privacy* slack does measurably degrade invariance (Sect. 12) — which is why it is zero by default.

The $O(\sqrt{T \log T})$ bound is conditional. It needs persistent excitation (A3), which a no-exploration policy does not guarantee: its own unvisited nodes keep their priors, and even a frequently visited node can see collinear features. The priority-safe probe restores node visitation over reachable candidates only, which is necessary but not sufficient for (A3), and the residual cost-prediction error in Sect. 9.5 is the visible trace of this gap. The per-step inequality of Thm. 1 holds without it.

Absolute outcomes are candidate-set properties. Sect. 9.4 shows this quantitatively: substituting a random menu moves both LEXI and the menu *oracle* from $\approx 1.5\%$ to $\approx 43.5\%$ SLO violations. The rule stays within 3×10^{-5} of the achievable optimum in both regimes, but the achievable optimum is set by the generator. The candidate menu therefore forms part of every absolute outcome reported here.

13.3 Scope of the privacy instantiation

Section 3.4 defines the metric. Its interpretation follows that definition.

The metric counts PII-tagged stages on publicly reachable nodes. It is a *topological* proxy and nothing more. It does not model data residency or jurisdiction, encryption at rest or in transit or the overhead either imposes, transit security between tiers, or — most importantly — the misconfiguration failure mode that accounts for the majority of real cloud data exposure, where storage is nominally private and its access policy is simply wrong [11]. A placement this metric scores as perfectly private can still leak, and the metric will not see it. Data residency is enforced separately as a hard pre-filter on the candidate set, reducing residency violations from 2.7% to 0% (Sect. 12). This treatment reflects the distinction between a prohibition and a ranked preference.

Known policy violations are excluded by the candidate generator. The ranked privacy coordinate therefore represents a preference among policy-compliant placements (“keep sensitive stages on-prem when the SLO allows”), which is a real operator statement and which a weighted sum prices and a lexicographic rule ranks. Readers who reject even that reading should substitute another second-priority objective — a tenancy-isolation count, a licence-tier count, a jurisdiction-crossing count — and every result in Sects. 9 and 12 carries over unchanged, because the rule never consults the coordinate’s meaning. This substitutability does not establish the proxy as a comprehensive privacy model.

13.4 Threats to validity

Construct validity. It is our own discrete-event simulator, and its parameters were chosen so that no tier dominates (Sect. 8.1). That is necessary for the question to be well-posed and it is also the strongest threat to construct validity, since a fleet with less tension would show smaller differences between rules. We mitigate by reporting every parameter and its rationale, by sweeping the SLO threshold, and by re-running on an independent configuration (config B) where the ordering holds — but we cannot rule out that some real fleets make the question uninteresting.

External validity: execution environment. Every number in this paper comes from simulation rather than a Kubernetes cluster, a DeathStarBench deployment, or physical edge hardware. Consequently, the evaluation provides no measurement of orchestration and control-plane overhead, the cost and duration of an actual stage migration (we count churn events rather than their price), telemetry propagation delay between the observed outcome and the model update, scheduler and admission-control interaction, container cold-start behaviour beyond the injected penalty, real network jitter and tail behaviour beyond the injected queueing noise, and failure or partition handling. The sub-0.1 ms decision time is an in-process measurement of the selection rule alone and should not be read as an end-to-end orchestration latency. Two of the simulator’s inputs — carbon intensity and arrival load — are driven from real public traces and every RQ2 conclusion survives that substitution (Sect. 11), which is evidence about the *signals* rather than about the execution environment. Hardware validation is the next external-validity step.

External validity: scale and concurrency. The primary configuration is **one** application: 7 nodes over 3 regions, a 6-stage DAG, 40 candidates per decision, 1,600 requests per run. Config B reaches 10 nodes and 8 stages. The scaling study reaches $K \leq 30$ stages and $C \leq 2000$ candidates and stays sub-millisecond, with an update cost that is $O(K)$ and independent of C (Sect. 12). These measurements characterize decision cost rather than decision quality at scale. The scale results do not cover continua of hundreds of nodes or for multiple concurrent applications. Independent applications sharing nodes is itself a violation of Assumption 1. We expect that LEXI would degrade there in the same way

Table 6 shows, and a contention-aware or admission-coupled extension would be required. This setting has not been evaluated.

Internal: seeds, tests, and reuse. The primary comparison uses 15 seeds with 95% bootstrap CIs and paired-bootstrap tests. Most sweeps use 6–8 seeds and the contention sweep uses 15, as stated at each result and is the reason we report sweep conclusions as orderings rather than as point estimates. All methods share one candidate set, one outcome model, and one seeded request stream. Differences are attributable to the decision rule. The flip side is that a defect in the shared model would bias all methods together — which is exactly what Table 6 shows happening under induced misspecification.

Baselines. We compare against tuned and untuned weighted sums, three scale-robust scalarisers sharing the tuned weights, an exact lexicographic LP, an SLO-constrained weighted sum, a single-objective RL controller, and two static heuristics. Recent continuum schedulers such as HuntMS, ClusterLess, and Festina optimize broader scheduling pipelines rather than the shared decision interface evaluated here. Section 14 states this comparability boundary and positions them as deployed systems rather than executed baselines.

13.5 Future work and artifact

The ranked rule is orthogonal to several extensions: an integrated candidate generator (partly explored in Sect. 12), a contention-aware world-model beyond the $M/G/1$ term, multi-application admission coupling, and a hardware cluster. Since carbon and load already come from real traces, a real cluster is the remaining external-validity step. The simulator, controller, baselines, and scripts are released: a single command verifies every committed number in this paper against the artifact, and the full experiments regenerate from the released scripts. Only the latency microbenchmark is machine-dependent.

14 Related Work

LEXI draws on four research lines. Table 10 compares whether each formulation represents an explicit priority order, avoids weight tuning, preserves decisions under monotone rescaling, learns without exploratory placement, and targets continuum placement. The paragraphs below explain the distinctions behind these entries.

Continuum and green service orchestration. Cloud-edge-serverless scheduling spans fog placement [5], edge offloading [35], sky computing [38], carbon-aware datacenter control [32,43], and the service-computing agenda [6]. Recent systems make this concrete: *HuntMS* [10] (first released as *Aceso*) geo-distributes microservices for carbon and cost under a latency-SLO *constraint*, *ClusterLess* [17] orchestrates serverless workflows against one *deadline*, and *Festina* [42] minimises

Table 10. LEXI versus multi-objective orchestration.  yes,  partial, blank no.

Approach	Priority order	No weight tuning	Scale inv.	Off-policy no explor.	Cont. place
Weighted-sum MORL [33,41]					
Scale-robust scal. [29,37]					
Constrained MDP/CPO [3,2]					
Lexicographic RL [19,36]					
LEXI (ours)					

serverless energy under a latency-SLO constraint. Each reduces the trade-off to one scalar under a constraint, inheriting its *scale*-sensitivity and unable to *rank* the unconstrained objectives. LEXI instead treats the four as a *ranked* bundle.

Why these three systems are not in our baseline panel. HuntMS, ClusterLess and Festina are **not baseline families here**: they are the closest *deployed systems* rather than baselines we ran, for two reasons. First, at the level this paper operates — the decision rule over a candidate menu — all three reduce to the same formulation: a hard feasibility constraint on one threshold objective, followed by minimisation of a single scalar (or a fixed-weight blend) over the feasible set. That formulation *is* in our panel, as *Constr-WS* (Sect. 9.3), which we run head-to-head on identical estimates and candidates and which turns out to be the strongest baseline we have. Re-implementing three systems that instantiate one rule would add no discriminating evidence. Second, each system is inseparable from a substrate our simulator does not provide — HuntMS from a geo-distributed cluster with its own carbon and latency telemetry, ClusterLess and Festina from a live serverless platform with real cold-start and autoscaling behaviour — so a port would strip out precisely the engineering those papers contribute and would misrepresent them. The consequence is a real limitation of our evaluation, recorded in Sect. 13: we compare against the *formulations* these systems adopt rather than against the systems, and a head-to-head on shared hardware remains future work. What we can say from the comparison we did run is narrow and specific: a constraint-plus-scalar formulation is priority-correct on the constrained objective and prices everything beneath it, which costs it 0.7% of decisions synthetically, 14.3% under real load, and its scale-invariance below the constraint (Table 4).

Privacy, QoS, goal programming, and NFV/SFC. Data-residency and privacy-aware computing steer sensitive computation to compliant nodes [46,14], and QoS-aware composition optimises over a catalogue [45], but both encode privacy or an SLO as a scalarised term or hard filter. LEXI instead makes privacy a *ranked* objective — with the scope caveat of Sect. 3.4, since a placement-level topological proxy captures neither residency nor the access-policy misconfigurations that dominate measured cloud data exposure [11], and hard prohibitions are better encoded as candidate pre-filters than as gradients. Ordering rather than pricing is classical: *pre-emptive* (lexicographic) goal programming optimises priority levels in sequence [8,39,28], and modern *hierarchical constrained control* solves the

same strict priority online as a cascade of quadratic programs with inequality goals at each level [23,16]. Both are offline, model-based ancestors of LEXI, assuming known per-level coefficients and re-solving to optimality each step. LEXI is their online, off-policy counterpart that *learns* outcomes counterfactually, decides in linear time, and needs a tracking guarantee they do not. Constraint-first NFV/SFC placement is the same static, known-cost embedding of priority [21].

Multi-objective, lexicographic, and constrained RL. Much of MORL [33] uses scalarisation [41], which prices the objectives. Scale-robust scalarisations (rank/min-max sums, Tchebycheff norms [29,37]) reduce scale-sensitivity but still *price* them. Scale-invariance alone does not recover a strict order (Sect. 9). A rank- or quantile-normalised *lexicographic* surrogate is instead a monotone reparameterisation of our own rule, which we measure to *exactly replicate* strict LEXI (100% agreement, Sect. 12). Lexicographic RL [19,36] also ranks objectives, but targets tabular control, with no service-graph model and no tracking guarantee for a *learned* world-model. Temporal-logic objectives also avoid hand-set weights and reduce optimally to average rewards [25], but they constrain behaviour over time, while LEXI orders objectives within one decision. The same priority-first pattern governs post-quantum profile selection, where admissibility gates run before cost ranking [24]. Our lower-priority tolerance echoes the slack of lexicographic MDPs [44] and thresholded lexicographic ordering [40], kept off the top priority. Constrained MDPs [3] and constrained policy optimisation [2] encode “meet the SLO first” as a hard constraint. This is the closest competitor to LEXI and it is a strong one: a hard constraint on a threshold objective is *exactly* the first level of a lexicographic rule. On the top priority the two formulations coincide, and our Constr-WS baseline — that constraint-then-scalarise decision rule applied per request rather than a learned constrained policy — duly matches LEXI there without tuning (Sect. 9.3). The difference is that a constrained formulation expresses one priority level and then a price: it is silent on ranking the *rest*, and inherits the weighted sum’s scale-sensitivity for every objective below the constraint, which we measure (Table 4). Lexicographic control generalises it to an arbitrary number of levels at the same asymptotic cost. Because latency has a tail [12], we guard the top priority with a safety *margin*, a chance-constraint surrogate [9]. LEXI adapts thresholded lexicographic control to service-DAG placement and proves a tracking bound for a learned, misspecified world-model that pure-RL analyses omit.

Learning-based scheduling and off-policy estimation. RL schedulers such as Decima [27] are *on-policy*: they execute a decision to observe its outcome, which in a continuum is a migration with a real cost. LEXI instead learns *off-policy* from a counterfactual model, estimating un-taken placements from executed ones, following counterfactual policy search [7], structural-causal models [31,4], and off-policy evaluation [13]. Our identification is *structural* (additive no-interference, SUTVA, Sect. 6), under which per-node regression suffices. We also implement *doubly-robust* [13] and contention-aware ($M/G/1$) variants that

partly lift SUTVA. To our knowledge, LEXI is the first continuum orchestrator to combine counterfactual off-policy learning, thresholded lexicographic control, and a misspecification-aware tracking guarantee.

15 Conclusion

We present LEXI, a cloud-edge-serverless orchestrator for objectives expressed as a strict priority order. The central design choice is simple: when operators mean “satisfy the SLO before optimising anything else,” the controller should encode that order directly instead of hiding it inside exchange rates. Our thresholded lexicographic rule applies this principle to predictions from an off-policy counterfactual outcome model. We instantiate the order with latency, privacy, carbon, and cost, while treating privacy as the topological preference defined in Sect. 3.4 rather than as a compliance verdict.

The experiments show why the distinction matters. Strict lexicographic control reaches the same target region as the hand-calibrated and constrained baselines without weight tuning, while only one of 84 weighted configurations meets the target. It remains unchanged under a $300\times$ privacy-scale swing, where the tuned and constrained baselines expose 1.85 and 1.84 of two PII stages. Under operational conflict, it records no priority inversions where priced scalarisers reach up to 37%, and its in-process decision takes <0.1 ms. Real carbon and load traces and an independent configuration preserve these comparisons.

These results establish the decision rule within one simulated application and its candidate menus. The same menu oracle follows LEXI in the $2\% \rightarrow 44\%$ SLO jump, showing that candidate generation determines what any selector can achieve. Likewise, contention weakens every method that shares the additive no-interference model. A hardware deployment with measured migration, telemetry, and multi-tenant effects is therefore the next step from decision-level evidence to end-to-end orchestration performance.

Acknowledgments. We acknowledge Ho Chi Minh City University of Technology (HCMUT), VNU-HCM for supporting this study.

References

1. Abbasi-Yadkori, Y., Pál, D., Szepesvári, C.: Improved algorithms for linear stochastic bandits. In: Proc. NIPS. pp. 2312–2320 (2011)
2. Achiam, J., Held, D., Tamar, A., Abbeel, P.: Constrained policy optimization. In: Proc. ICML. pp. 22–31 (2017)
3. Altman, E.: Constrained Markov Decision Processes. Chapman and Hall/CRC (1999)
4. Bareinboim, E., Pearl, J.: Causal inference and the data-fusion problem. Proc. Natl. Acad. Sci. **113**(27), 7345–7352 (2016)
5. Bonomi, F., Milito, R., Zhu, J., Addepalli, S.: Fog computing and its role in the Internet of things. In: Proc. ACM MCC. pp. 13–16 (2012)

6. Bouguettaya, A., Singh, M., Huhns, M., Sheng, Q.Z., Dong, H., Yu, Q., Neiat, A.G., Mistry, S., Benatallah, B., Medjahed, B., Ouzzani, M., Casati, F., Liu, X., Wang, H., Georgakopoulos, D., Chen, L., Nepal, S., Malik, Z., Erradi, A., Wang, Y., Blake, B., Dustdar, S., Leymann, F., Papazoglou, M.: A service computing manifesto: The next 10 years. *Commun. ACM* **60**(4), 64–72 (2017)
7. Buesing, L., Weber, T., Zwols, Y., Racanière, S., Guez, A., Lespiau, J.B., Heess, N.: Woulda, coulda, shoulda: Counterfactually-guided policy search. In: *Proc. ICLR* (2019)
8. Charnes, A., Cooper, W.W.: Goal programming and multiple objective optimizations: Part 1. *Eur. J. Oper. Res.* **1**(1), 39–54 (1977)
9. Chow, Y., Ghavamzadeh, M., Janson, L., Pavone, M.: Risk-constrained reinforcement learning with percentile risk criteria. *J. Mach. Learn. Res.* **18**(167), 1–51 (2018)
10. Christofidi, G., Álvarez-Terribas, F., Roumpos, I., Kourtellis, N., Omaña Iglesias, J., Doudali, T.D.: HuntMS: A framework for microservice geo-distribution for carbon and cost reduction. *arXiv preprint arXiv:2603.10768v2* (2026), v1 titled “Aceso”
11. Continella, A., Polino, M., Pogliani, M., Zanero, S.: There’s a hole in that bucket! a large-scale analysis of misconfigured S3 buckets. In: *Proc. 34th Annual Computer Security Applications Conference (ACSAC)*. pp. 702–711 (2018). <https://doi.org/10.1145/3274694.3274736>
12. Dean, J., Barroso, L.A.: The tail at scale. *Commun. ACM* **56**(2), 74–80 (2013)
13. Dudík, M., Langford, J., Li, L.: Doubly robust policy evaluation and learning. In: *Proc. ICML*. pp. 1097–1104 (2011)
14. Dwork, C., Roth, A.: The algorithmic foundations of differential privacy. *Found. Trends Theor. Comput. Sci.* **9**(3–4), 211–407 (2014)
15. Ehrgott, M.: *Multicriteria Optimization*. Springer, 2nd edn. (2005)
16. Escande, A., Mansard, N., Wieber, P.B.: Hierarchical quadratic programming: Fast online humanoid-robot motion generation. *Int. J. Robot. Res.* **33**(7), 1006–1028 (2014)
17. Farahani, R., Colosi, M., Murturi, I., Nastic, S., Villari, M., Dustdar, S., Prodan, R.: ClusterLess: Deadline-aware serverless workflow orchestration on federated edge clusters. In: *Proc. IEEE ICDCS* (2026), *arXiv:2605.04310*
18. Fishburn, P.C.: Lexicographic orders, utilities and decision rules: A survey. *Manag. Sci.* **20**(11), 1442–1471 (1974)
19. Gábor, Z., Kalmár, Z., Szepesvári, C.: Multi-criteria reinforcement learning. In: *Proc. ICML*. pp. 197–205 (1998)
20. Gan, Y., Zhang, Y., Cheng, D., Shetty, A., Rathi, P., Katarki, N., Bruno, A., Hu, J., Ritchken, B., Jackson, B., Hu, K., Pancholi, M., He, Y., Clancy, B., Colen, C., Wen, F., Leung, C., Wang, S., Zaruvinsky, L., Espinosa, M., Lin, R., Liu, Z., Padilla, J., Delimitrou, C.: An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In: *Proc. ASPLOS*. pp. 3–18 (2019)
21. Herrera, J.G., Botero, J.F.: Resource allocation in NFV: A comprehensive survey. *IEEE Trans. Netw. Serv. Manag.* **13**(3), 518–532 (2016)
22. Jonas, E., Schleier-Smith, J., Sreekanti, V., Tsai, C.C., Khandelwal, A., Pu, Q., Shankar, V., Carreira, J., Krauth, K., Yadwadkar, N., Gonzalez, J.E., Popa, R.A., Stoica, I., Patterson, D.A.: Cloud programming simplified: A Berkeley view on serverless computing. *arXiv preprint arXiv:1902.03383* (2019)

23. Kanoun, O., Lamiroux, F., Wieber, P.B.: Kinematic control of redundant manipulators: Generalizing the task-priority framework to inequality task. *IEEE Trans. Robot.* **27**(4), 785–792 (2011)
24. Le, X.B., Tran-Truong, P.T., Nguyen, D., Nguyen, N.K.T., Ha, X.S.: QuantumTrustKG: Provenance-aware orchestration for quantum-safe microservices. In: *Proc. ICSOC. LNCS*, Springer (2026), to appear
25. Le, X.B., Wagner, D., Witzman, L., Rabinovich, A., Ong, L.: Reinforcement learning with LTL and ω -regular objectives via optimality-preserving translation to average rewards. In: *Advances in Neural Information Processing Systems 37 (NeurIPS)* (2024). <https://doi.org/10.52202/079017-3718>
26. Luo, S., Xu, H., Lu, C., Ye, K., Xu, G., Zhang, L., Ding, Y., He, J., Xu, C.: Characterizing microservice dependency and performance: Alibaba trace analysis. In: *Proc. ACM SoCC*. pp. 412–426 (2021)
27. Mao, H., Schwarzkopf, M., Venkatakrisnan, S.B., Meng, Z., Alizadeh, M.: Learning scheduling algorithms for data processing clusters. In: *Proc. ACM SIGCOMM*. pp. 270–288 (2019)
28. Marler, R.T., Arora, J.S.: Survey of multi-objective optimization methods for engineering. *Struct. Multidiscip. Optim.* **26**(6), 369–395 (2004)
29. Miettinen, K.: *Nonlinear Multiobjective Optimization*, International Series in Operations Research & Management Science, vol. 12. Kluwer Academic Publishers (1999)
30. National Energy System Operator: Carbon intensity API. <https://carbonintensity.org.uk/> (2026), regional half-hourly forecasts; accessed 24 September 2026
31. Pearl, J.: *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edn. (2009)
32. Radovanović, A., Koningstein, R., Schneider, I., Chen, B., Duarte, A., Roy, B., Xiao, D., Haridasan, M., Hung, P., Care, N., Talukdar, S., Mullen, E., Smith, K., Cottman, M., Cirne, W.: Carbon-aware computing for datacenters. *IEEE Trans. Power Syst.* **38**(2), 1270–1280 (2023)
33. Roijers, D.M., Vamplew, P., Whiteson, S., Dazeley, R.: A survey of multi-objective sequential decision-making. *J. Artif. Intell. Res.* **48**, 67–113 (2013)
34. Shahradd, M., Fonseca, R., Goiri, Í., Chaudhry, G., Batum, P., Cooke, J., Laureano, E., Tresness, C., Russinovich, M., Bianchini, R.: Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In: *Proc. USENIX ATC*. pp. 205–218 (2020)
35. Shi, W., Cao, J., Zhang, Q., Li, Y., Xu, L.: Edge computing: Vision and challenges. *IEEE Internet Things J.* **3**(5), 637–646 (2016)
36. Skalse, J., Hammond, L., Griffin, C., Abate, A.: Lexicographic multi-objective reinforcement learning. In: *Proc. IJCAI*. pp. 3430–3436 (2022)
37. Steuer, R.E.: *Multiple Criteria Optimization: Theory, Computation, and Application*. John Wiley & Sons (1986)
38. Stoica, I., Shenker, S.: From cloud computing to sky computing. In: *Proc. HotOS*. pp. 26–32 (2021)
39. Tamiz, M., Jones, D., Romero, C.: Goal programming for decision making: An overview of the current state-of-the-art. *Eur. J. Oper. Res.* **111**(3), 569–581 (1998)
40. Tercan, A., Prabhu, V.S.: Thresholded lexicographic ordered multiobjective reinforcement learning. In: *Proc. ECAI*. pp. 3006–3014 (2024)
41. Van Moffaert, K., Nowé, A.: Multi-objective reinforcement learning using sets of pareto dominating policies. *J. Mach. Learn. Res.* **15**, 3663–3692 (2014)

- 42. Wang, T., Rattihalli, G., Dhakal, A., Shangguan, L., Milojicic, D.: Energy-aware scheduling for serverless LLM serving on shared GPUs. arXiv preprint arXiv:2606.30391 (2026)
- 43. Wiesner, P., Behnke, I., Scheinert, D., Gontarska, K., Thamsen, L.: Let's wait awhile: How temporal workload shifting can reduce carbon emissions in the cloud. In: Proc. Middleware. pp. 260–272 (2021)
- 44. Wray, K.H., Zilberstein, S., Mouaddib, A.I.: Multi-objective MDPs with conditional lexicographic reward preferences. In: Proc. AAAI. pp. 3418–3424 (2015)
- 45. Zeng, L., Benatallah, B., Ngu, A.H.H., Dumas, M., Kalagnanam, J., Chang, H.: QoS-aware middleware for web services composition. IEEE Trans. Softw. Eng. **30**(5), 311–327 (2004)
- 46. Zhang, K., Zhou, X., Chen, Y., Wang, X., Ruan, Y.: Sedic: Privacy-aware data intensive computing on hybrid clouds. In: Proc. ACM CCS. pp. 515–526 (2011)